

05. Local Features Detection

Jae-Young Choi

Sungkyunkwan University

Outline



- Local Features
- Feature Detectors
 - Edge, Line, Corner, Blob

Local Features

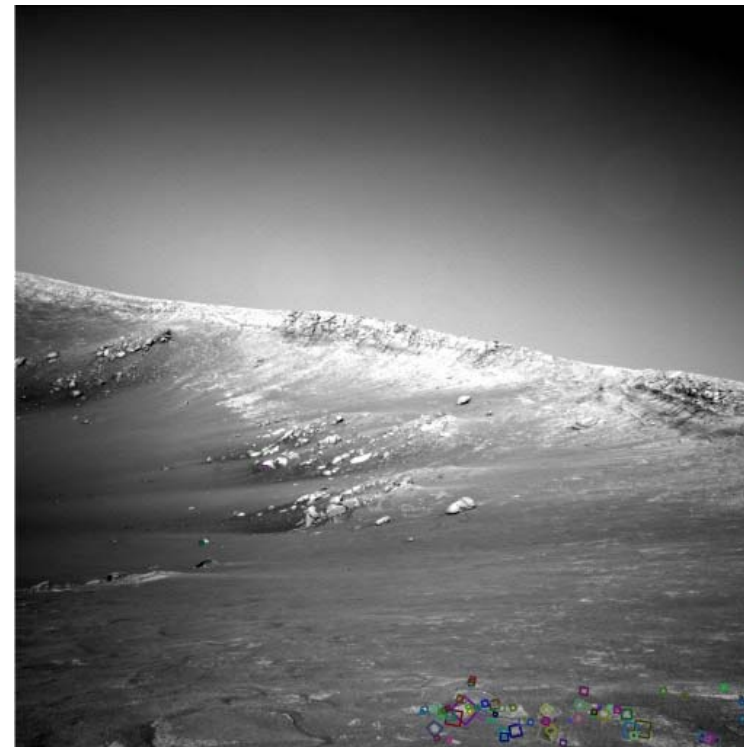
Why detect/extract features?



Source: S. Seitz

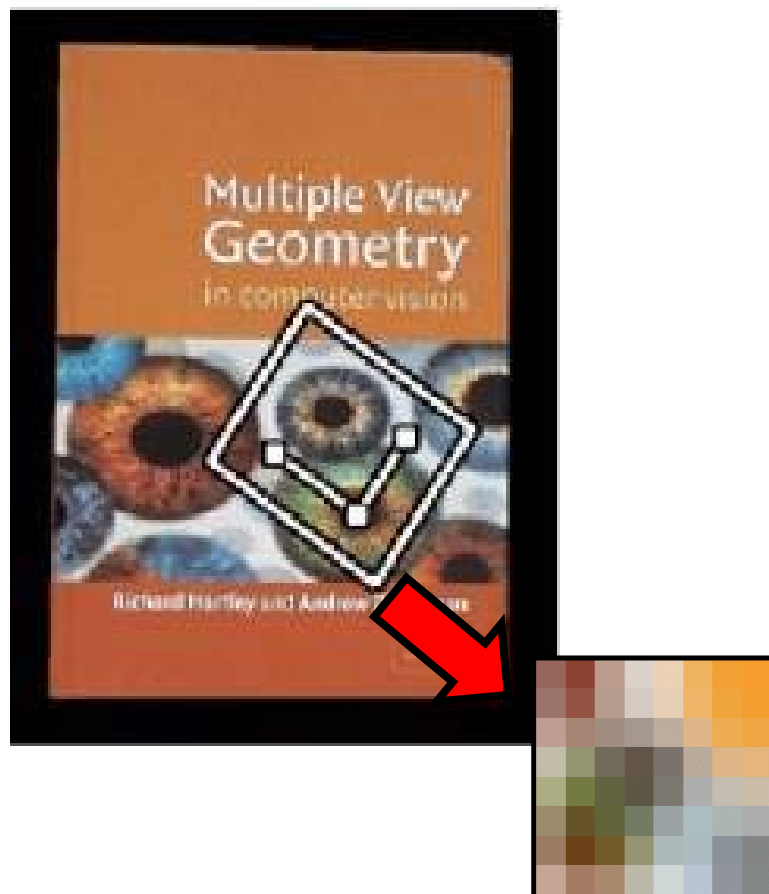
Step 1: extract features → Step 2: match features → Step 3: align images

Why detect/extract features?



NASA Mars Rover images with SIFT feature matches

Why detect/extract features?



Why detect/extract features?

- More motivation...
 - Image matching
 - Image alignment (e.g., mosaics)
 - 3D reconstruction
 - Motion tracking (e.g. for AR)
 - Object recognition
 - Image retrieval
 - Robot navigation
 - other



Why detect/extract features?

- Advantages of local features
 - Locality
 - features are local, so robust to occlusion and clutter
 - Quantity
 - hundreds or thousands in a single image
 - Distinctiveness
 - can differentiate a large database of objects
 - Efficiency
 - real-time performance achievable

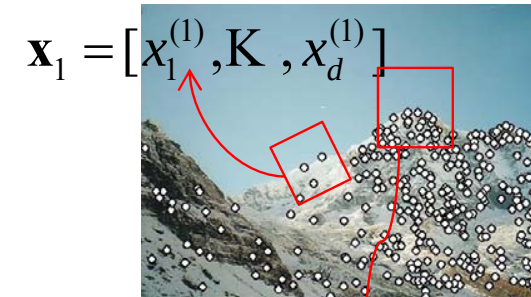
How detect/extract features?

- Local features: main components

Detection: Identify the interest points

- Description:** Extract vector feature descriptor surrounding each interest point.

- Matching:** Determine correspondence between descriptors in two views



$$\mathbf{x}_1 = [x_1^{(1)}, K, x_d^{(1)}]$$

$$\mathbf{x}_2 = [x_1^{(2)}, K, x_d^{(2)}]$$



How detect/extract features?

Local, meaningful, detectable parts of the image.

■ Edge detection

- using derivatives (image gradient)
→ Difference operators
- Prewitt operator, Sobel operator,
Robert's cross operator

■ Line detection

- using edge detection & non-maximum suppression
- connected components on edge pixels with similar orientation
- Canny operator, Hough Transform

■ Corner detection

- using a large change of intensity
in any direction
- Harris detector

■ Blob detection

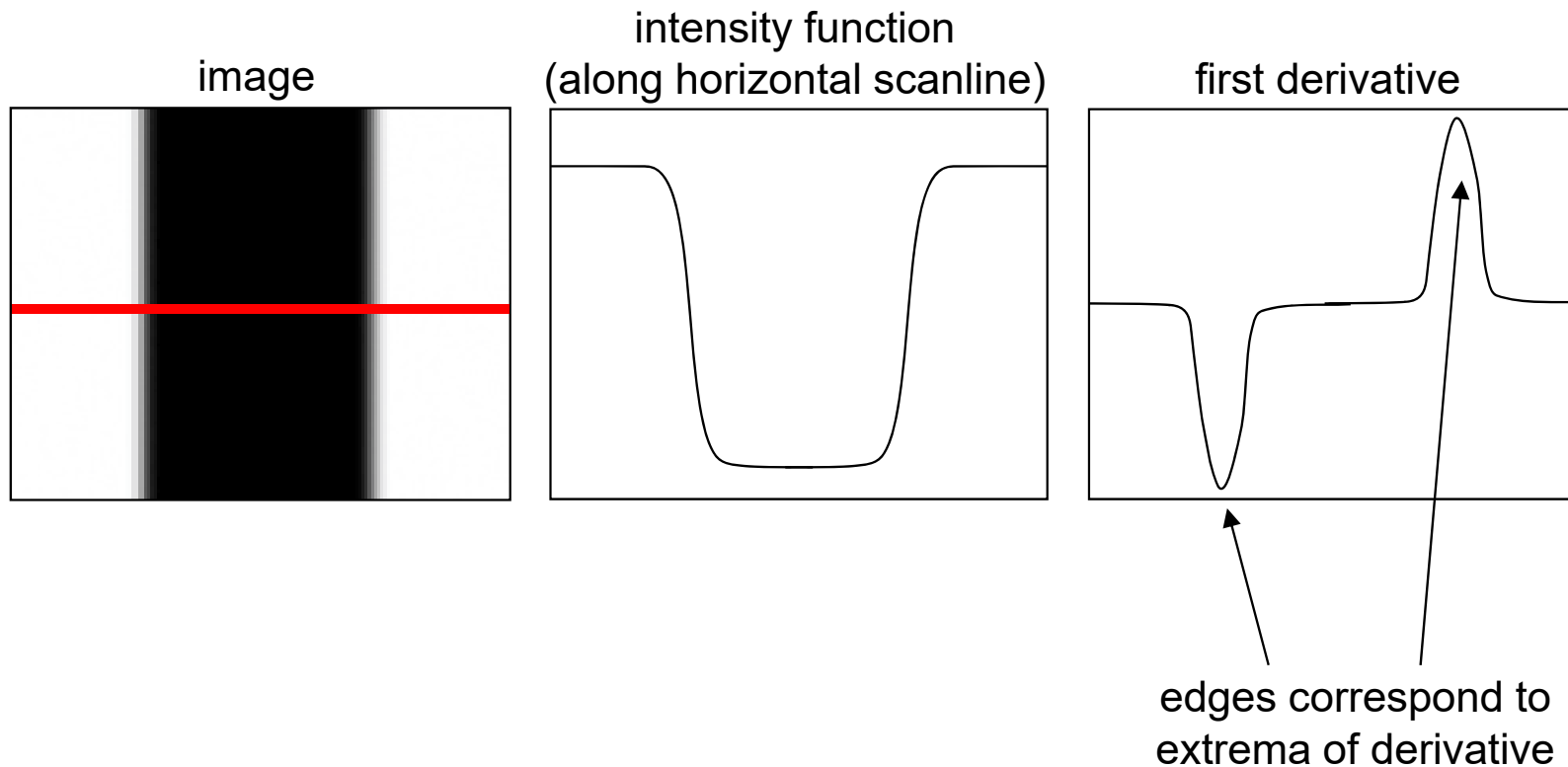
- Finding characteristic region size
- Laplacian of Gaussian

Edge detection

Detecting edges

- Characterizing edges

- An edge is a place of **rapid change** in the image intensity function



Detecting edges

- How can we differentiate a digital image $F[x,y]$?
 - Option 1: reconstruct a continuous image, f , then compute the derivative
 - Option 2: take discrete derivative (finite difference)

$$\frac{\partial f}{\partial x}[x, y] \approx F[x + 1, y] - F[x, y]$$

- How would you implement this as a linear filter?

$$\frac{\partial f}{\partial x} : \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

H_x

$$\frac{\partial f}{\partial y} : \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

H_y

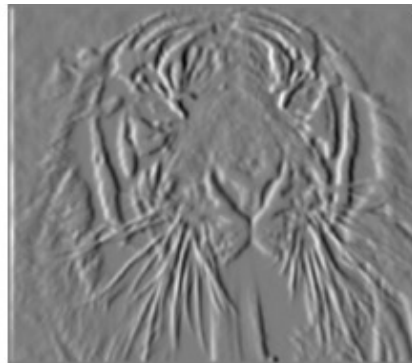
Detecting edges

- Partial derivatives of an image

$$\frac{\partial f(x, y)}{\partial x}$$

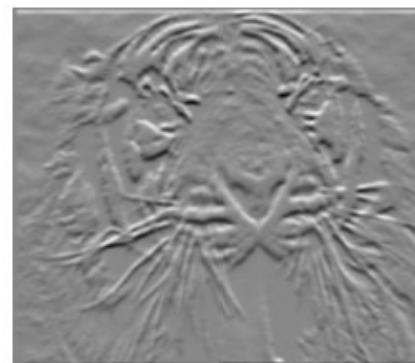
-1	1
----	---

$$-f(x, y) + f(x + 1, y)$$



$$\frac{\partial f(x, y)}{\partial y}$$

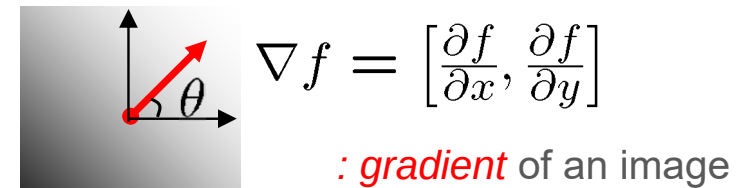
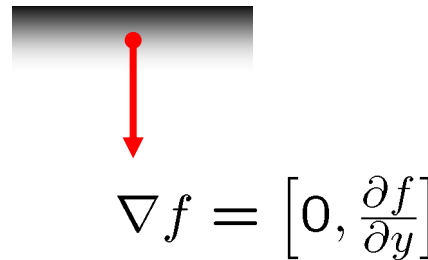
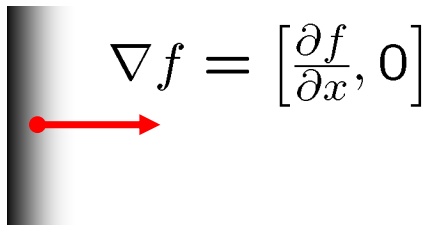
-1
1



Detecting edges

- Consider the image gradient

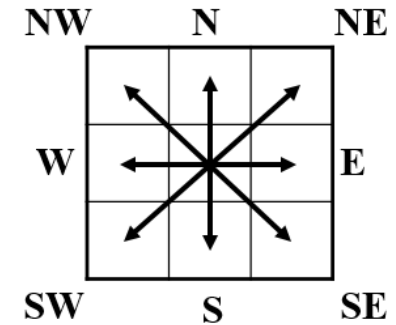
- The gradient points in the direction of most rapid increase in intensity



- The edge strength is given by the *gradient magnitude*: $\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$
- The *gradient direction* is given by: $\theta = \tan^{-1} \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$

Detecting edges

- Compass gradient masks
 - Use 8 masks aligned with the usual compass directions
 - Select largest response (magnitude)
 - Orientation is the direction associated with the largest response
- Gradient magnitude
 - is given by the maximum response
- Gradient direction
 - is the compass direction of the maximum response



perpetrator

			N																														
NW	<table><tr><td>2</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>-1</td></tr><tr><td>0</td><td>-1</td><td>-2</td></tr></table>	2	1	0	1	0	-1	0	-1	-2	<table><tr><td>1</td><td>2</td><td>1</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>-1</td><td>-2</td><td>-1</td></tr></table>	1	2	1	0	0	0	-1	-2	-1		<table><tr><td>0</td><td>1</td><td>2</td></tr><tr><td>-1</td><td>0</td><td>1</td></tr><tr><td>-2</td><td>-1</td><td>0</td></tr></table>	0	1	2	-1	0	1	-2	-1	0	NE	
2	1	0																															
1	0	-1																															
0	-1	-2																															
1	2	1																															
0	0	0																															
-1	-2	-1																															
0	1	2																															
-1	0	1																															
-2	-1	0																															
	W																																
	<table><tr><td>1</td><td>0</td><td>-1</td></tr><tr><td>2</td><td>0</td><td>-2</td></tr><tr><td>1</td><td>0</td><td>-1</td></tr></table>	1	0	-1	2	0	-2	1	0	-1			<table><tr><td>-1</td><td>0</td><td>1</td></tr><tr><td>-2</td><td>0</td><td>2</td></tr><tr><td>-1</td><td>0</td><td>1</td></tr></table>	-1	0	1	-2	0	2	-1	0	1	E										
1	0	-1																															
2	0	-2																															
1	0	-1																															
-1	0	1																															
-2	0	2																															
-1	0	1																															
	SW																																
	<table><tr><td>0</td><td>-1</td><td>-2</td></tr><tr><td>1</td><td>0</td><td>-1</td></tr><tr><td>2</td><td>1</td><td>0</td></tr></table>	0	-1	-2	1	0	-1	2	1	0	<table><tr><td>-1</td><td>-2</td><td>-1</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>2</td><td>1</td></tr></table>	-1	-2	-1	0	0	0	1	2	1		<table><tr><td>-2</td><td>-1</td><td>0</td></tr><tr><td>-1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>2</td></tr></table>	-2	-1	0	-1	0	1	0	1	2	SE	
0	-1	-2																															
1	0	-1																															
2	1	0																															
-1	-2	-1																															
0	0	0																															
1	2	1																															
-2	-1	0																															
-1	0	1																															
0	1	2																															
			S																														

Detecting edges

■ Steps in edge detection

Filtering



Enhancement

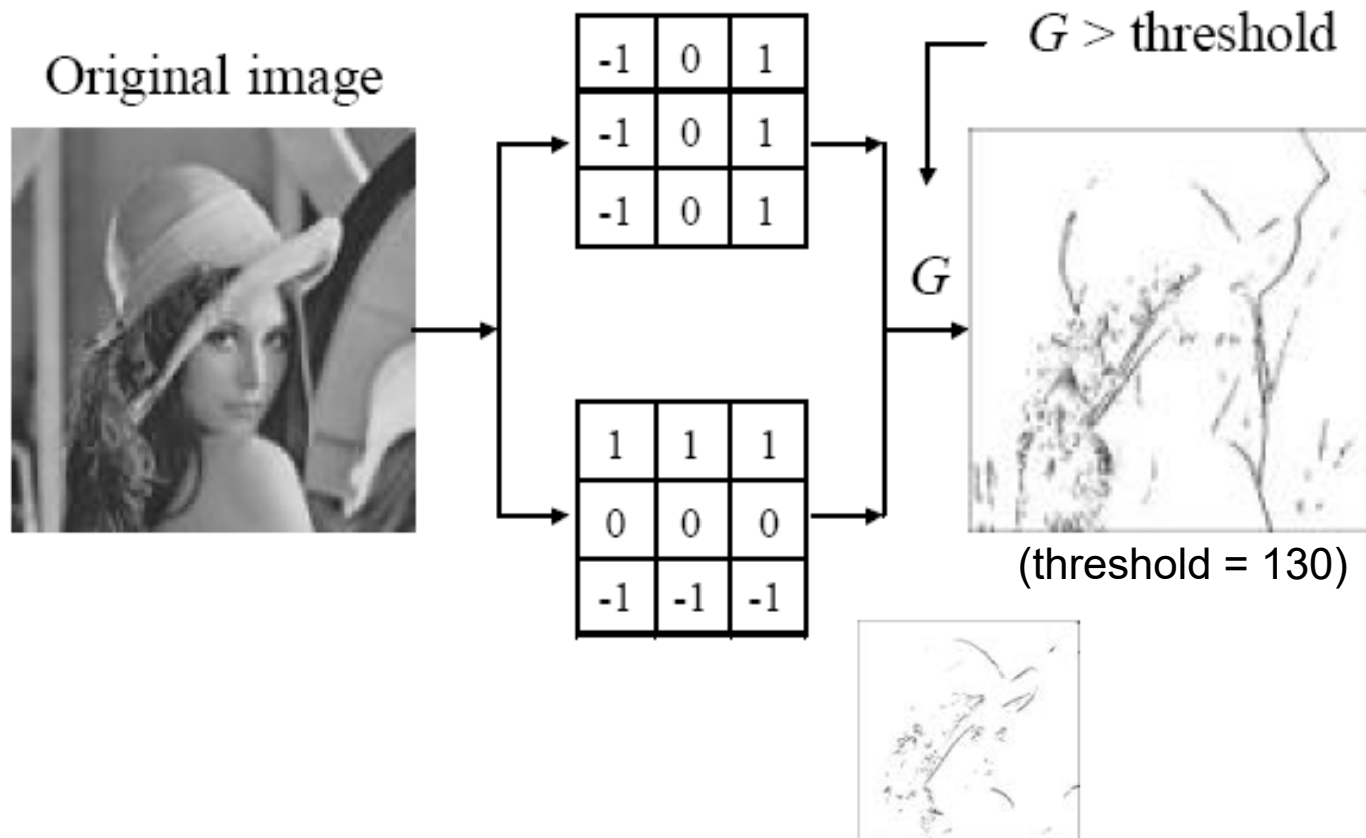


Detection

- To improve the performance of an edge detector with respect to noise
- Tradeoff between edge strength and noise reduction
- Emphasizes pixel where there is a significant change in local intensity value
- Find strong edge content

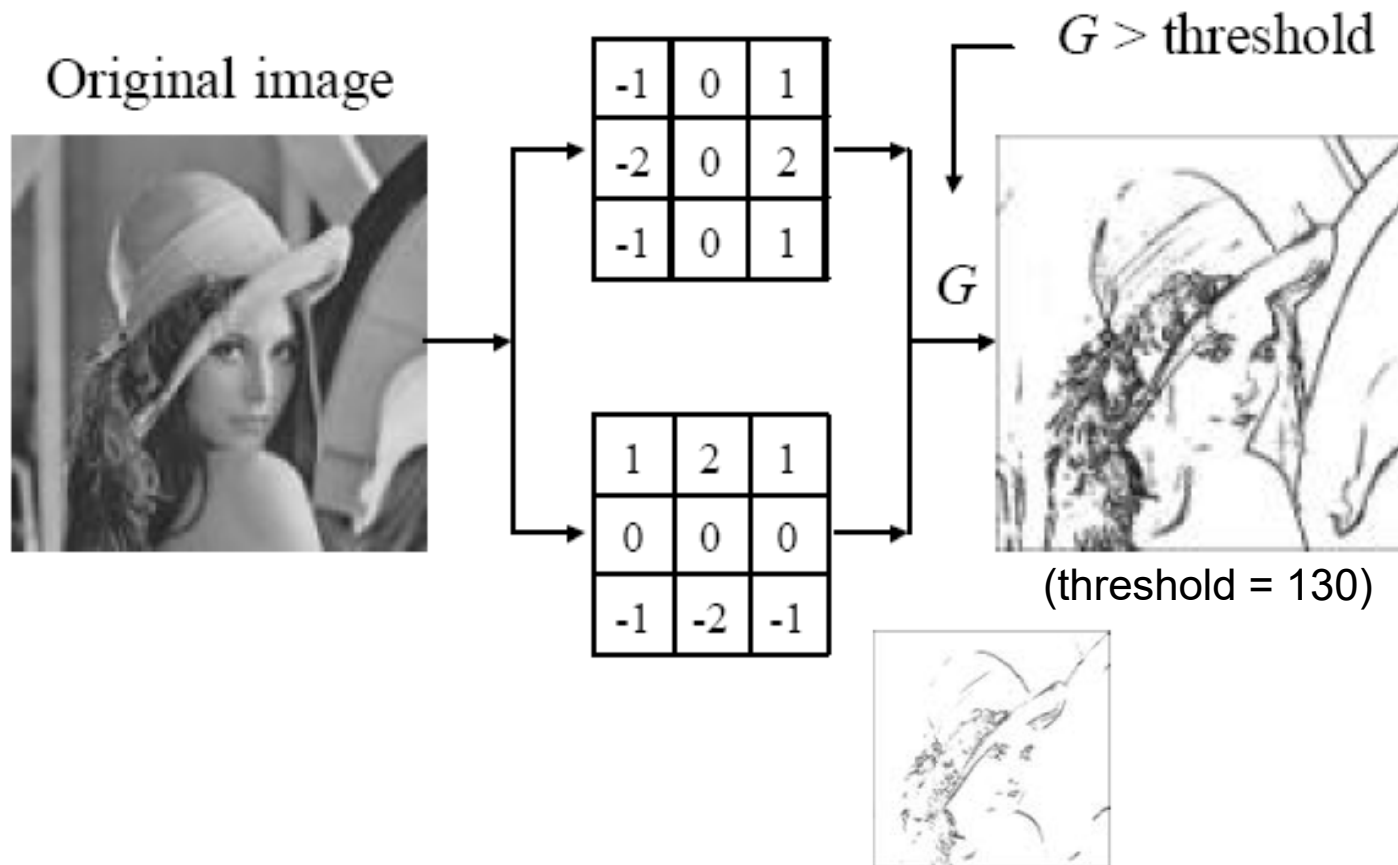
Difference operators

- Prewitt operator



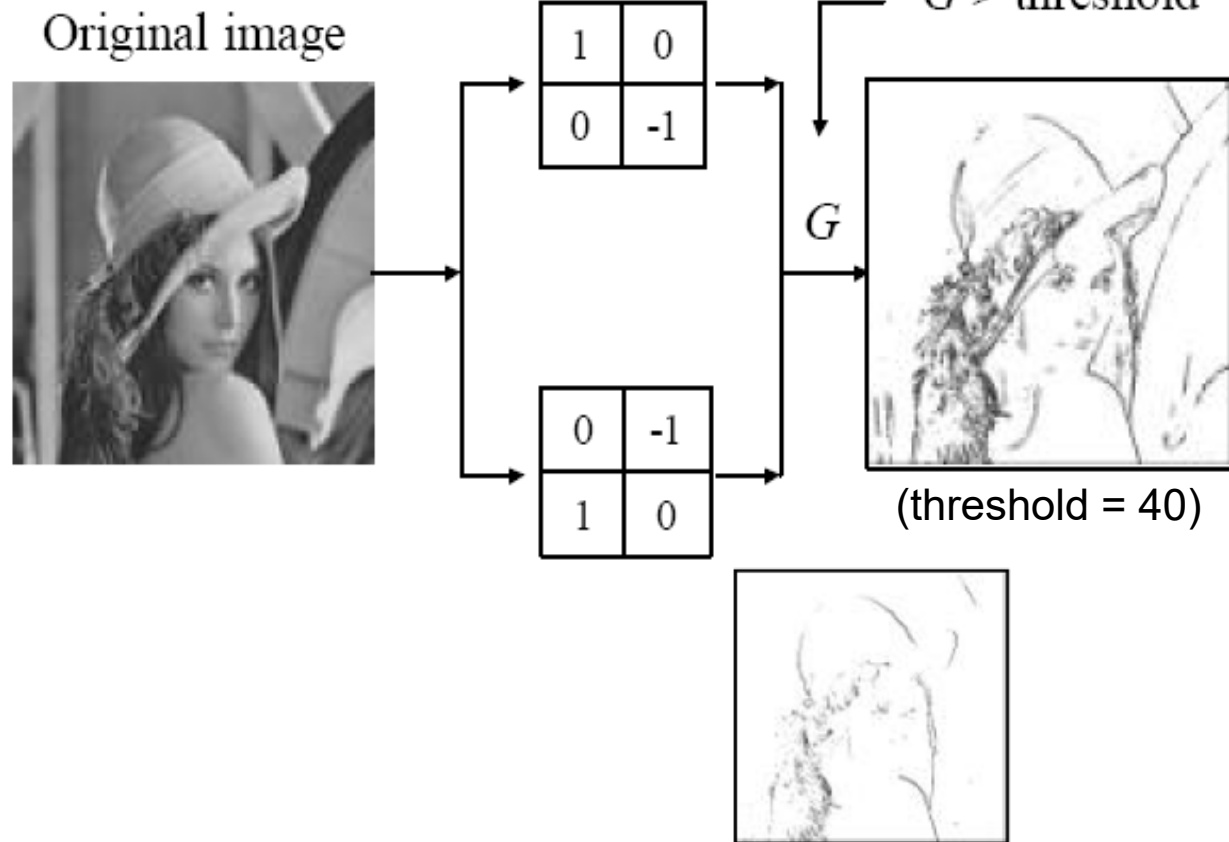
Difference operators

- Sobel operator



Difference operators

- Robert's cross operator



Line detection

Canny Edge Detector

- There are three major issues in edge detection:
 - 1) The gradient magnitude at different scales is different;
which should we choose?
 - 2) The gradient magnitude is large along thick trail;
how do we identify the significant points?
 - 3) How do we link the relevant points up into curves?

Canny Edge Detector

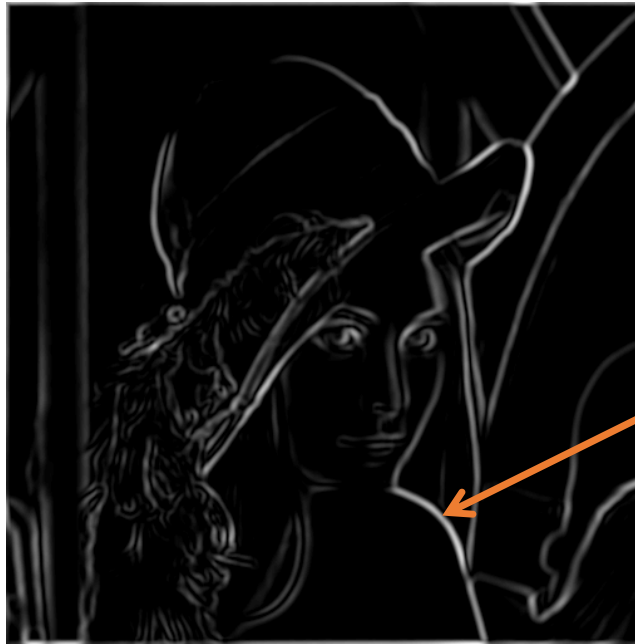
1. Filter image with derivative of Gaussian
2. Find magnitude and orientation of gradient
3. Non-maximum suppression
4. Linking and thresholding (hysteresis):
 - Define two thresholds: low and high
 - Use the high threshold to start edge curves and the low threshold to continue them

Canny Edge Detector

1. Filter image with derivative of Gaussian
2. Find magnitude and orientation of gradient



Norm of the gradient



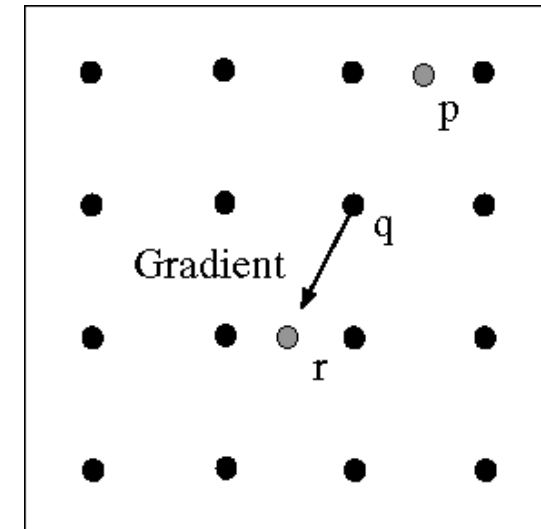
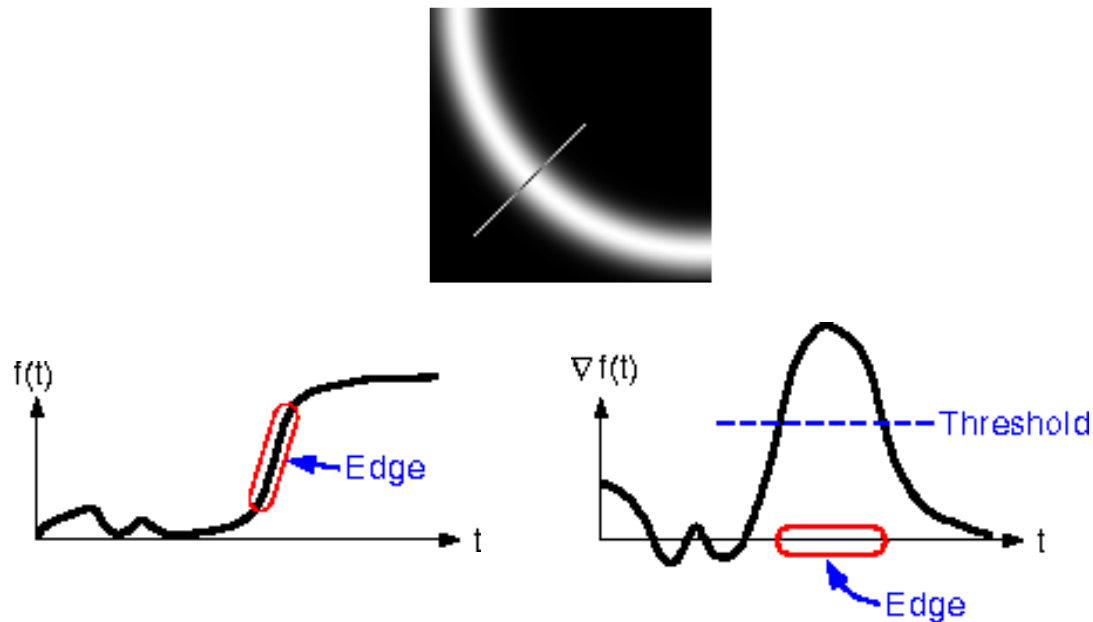
Thresholding

- We may have wide ridges around the local maxima
- How to turn these thick regions of the gradient into curves?

Canny Edge Detector

3. Non-maximum suppression

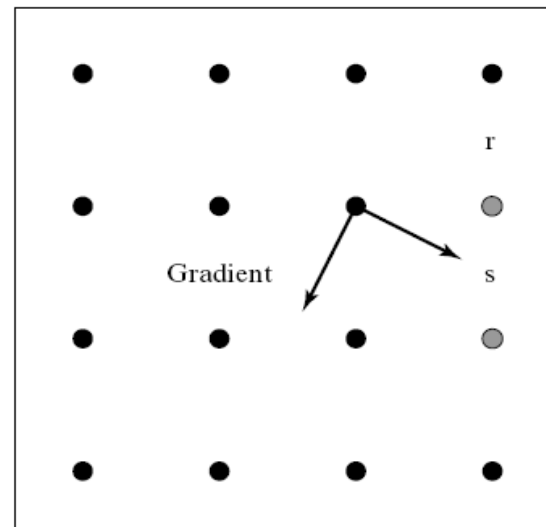
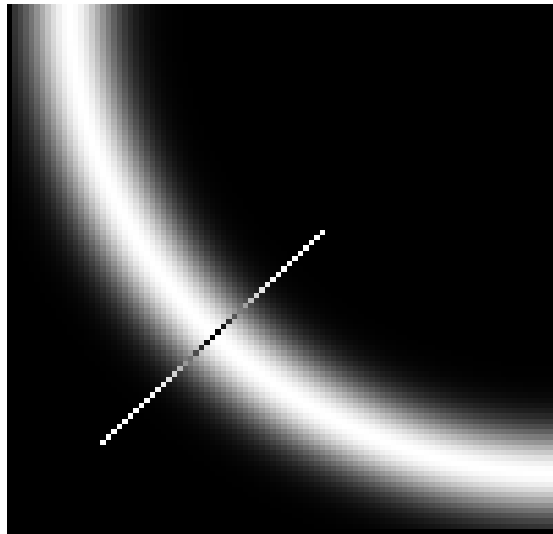
- Check if a pixel is local maximum along gradient direction
- At q , we have a maximum if the value is larger than those at both p and at r



Canny Edge Detector

3. Non-maximum suppression

- Assume the marked point is an edge point
- Then, construct the tangent to the edge curve and use this to predict the next points (here, either r or s)



Canny Edge Detector

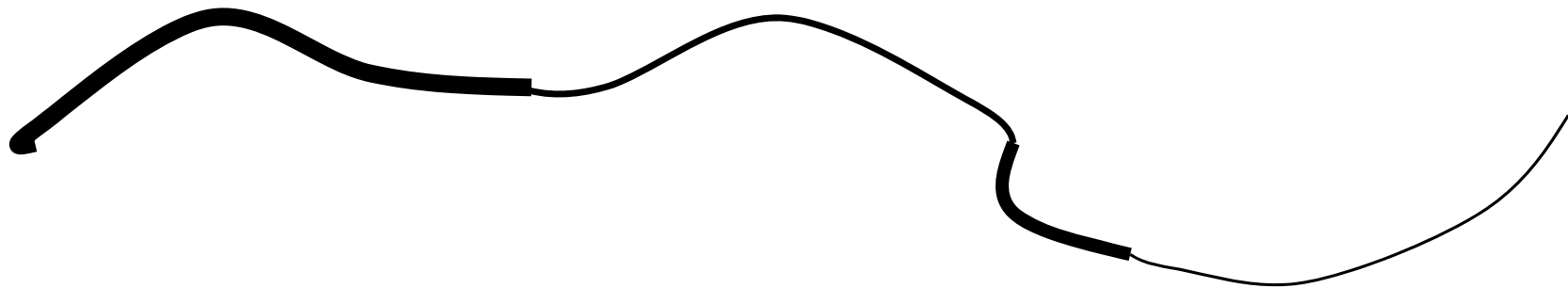
3. Non-maximum suppression

- Another problem: pixels along this edge didn't survive the thresholding



4. Hysteresis thresholding

- Check that maximum value of gradient value is sufficiently large
- Define two thresholds: low and high
- Use a high threshold to start edge curves and a low threshold to continue them



Canny Edge Detector

https://docs.opencv.org/master/da/d22/tutorial_py_canny.html

<https://youtu.be/b-K-vA1eiMA>

4. Hysteresis thresholding



original image



high threshold
(strong edges)



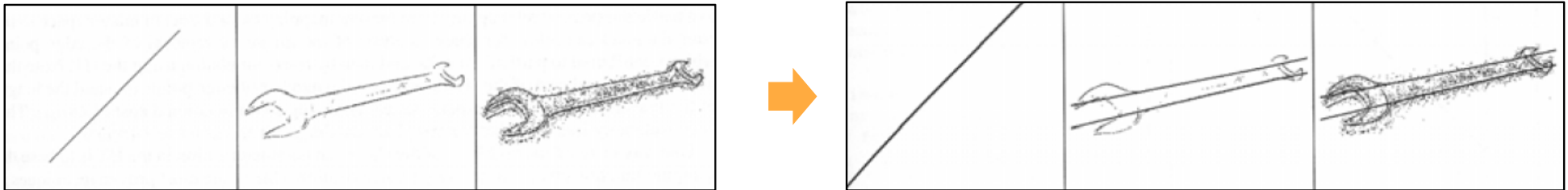
low threshold
(weak edges)



hysteresis threshold

Hough Transform

- is a technique which can be used to isolate features of a particular shape within an image
- The goal is to find the location of lines in images
- can be used to detect lines, circles or other parametric curves
- It can give robust detection under noise and partial occlusion
- Representing a line
 - The $[a, b]$ representation
 - The $[\rho, \theta]$ representation



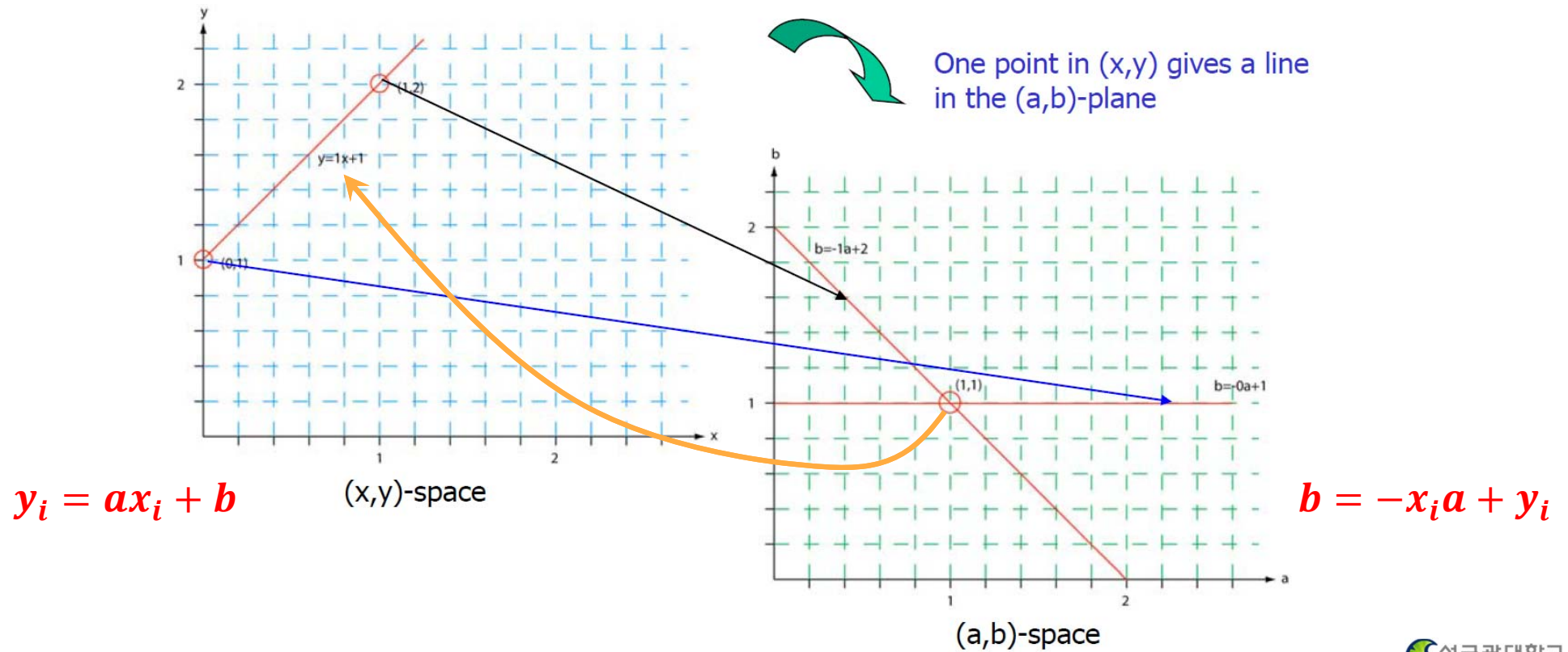
Hough Transform

■ The $[a, b]$ representation

- Assume that we have performed some edge detection, and a thresholding of the edge magnitude image
- Thus, we have n pixels that may partially describe the boundary of some objects
- We wish to find sets of pixels that make up straight lines
- Regard a point (x_i, y_i) and a straight line $y_i = ax_i + b$
 - There are many lines passing through the point (x_i, y_i)
 - Common to them is that they satisfy the equation for some set of parameters (a, b)
- The equation can obviously be rewritten as follows: $b = -x_i a + y_i$
- We now consider x and y as parameters and a and b as variables
- So, a single point in xy -space gives a line in (a, b) space
 - Another point (x_i, y_i) will give rise to another line in (a, b) space

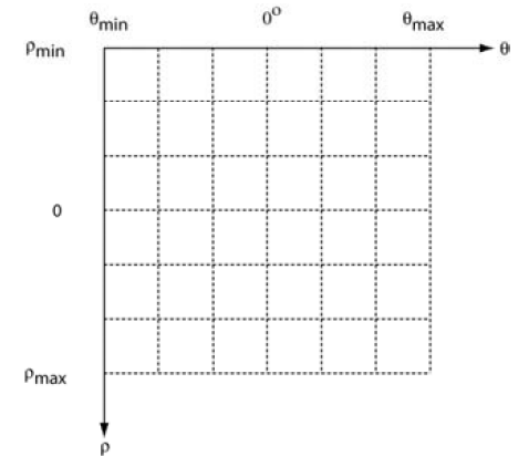
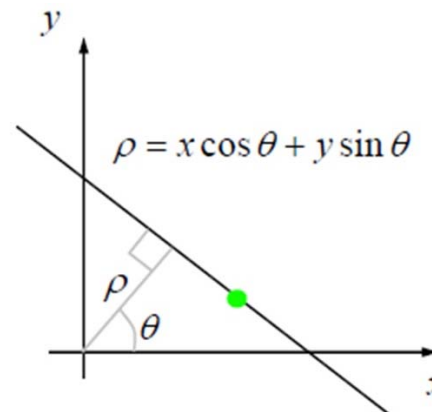
Hough Transform

- The $[a, b]$ representation



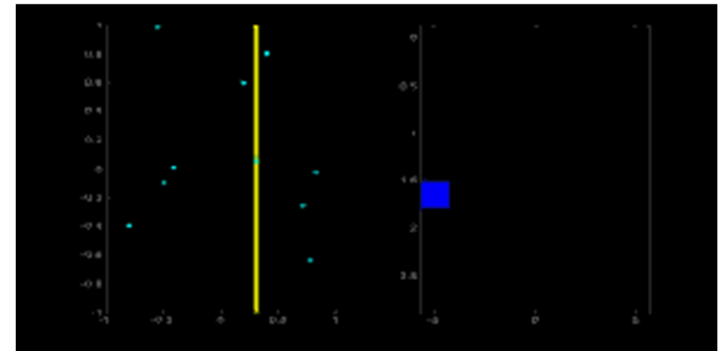
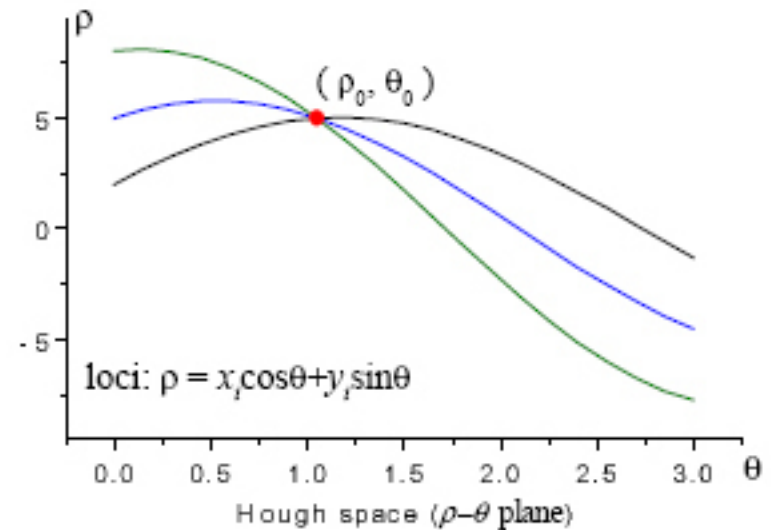
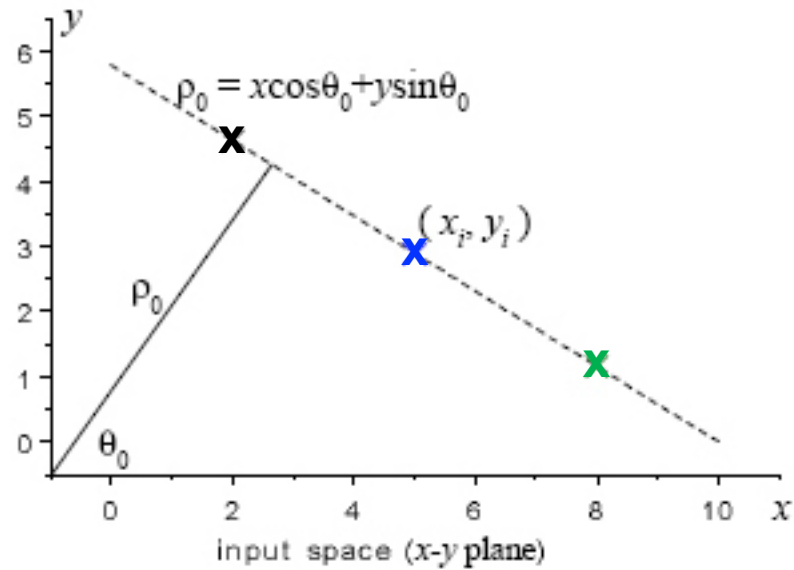
Hough Transform

- The $[\rho, \theta]$ representation (polar representation)
 - The polar (also called normal) representation of straight lines is
$$x \cos \theta + y \sin \theta = \rho$$
 - Each point (x_i, y_i) in the xy -plane gives a sinusoid in the $\rho\theta$ -plane
 - M colinear point lying on the line $x_i \cos \theta + y_i \sin \theta = \rho$ will give M curves that intersect at (ρ_i, θ_j) in the parameter plane
 - Local maxima give significant lines



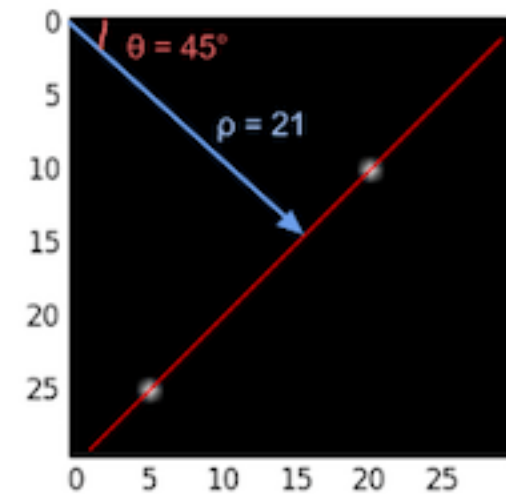
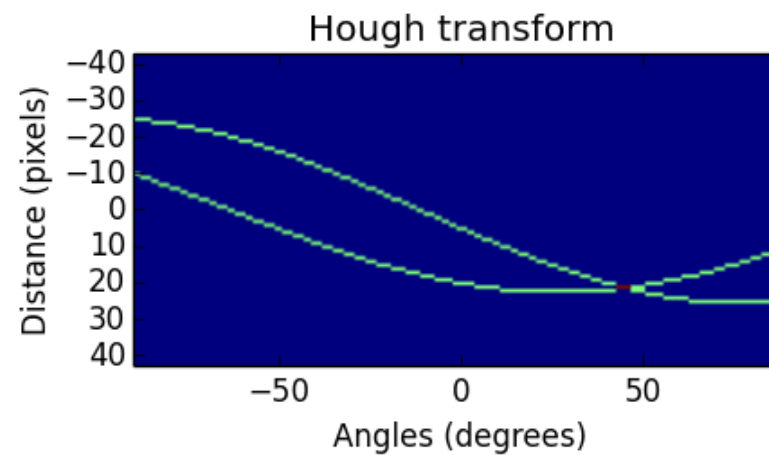
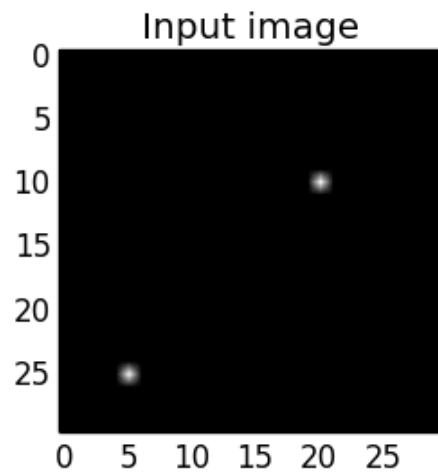
Hough Transform

- The $[\rho, \theta]$ representation



Hough Transform

- The $[\rho, \theta]$ representation



Hough Transform

https://docs.opencv.org/master/d6/d10/tutorial_py_houghlines.html

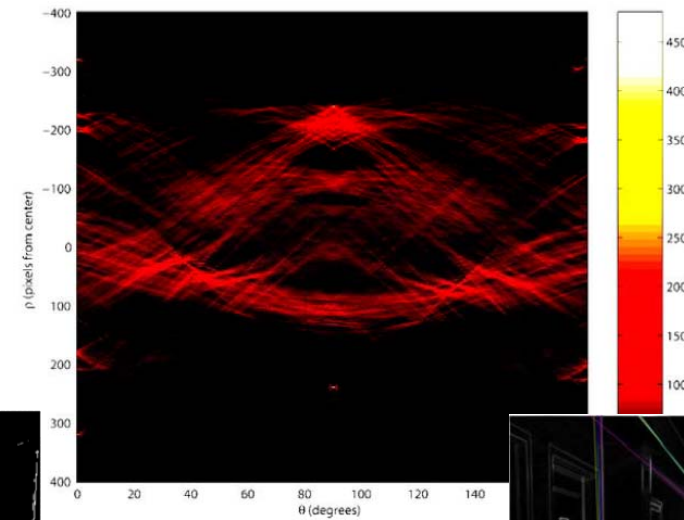
- The $[\rho, \theta]$ representation



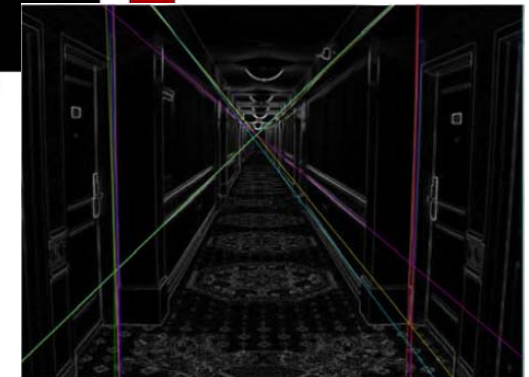
Input image



Sobel edge detection
& thresholding



Accumulator matrix

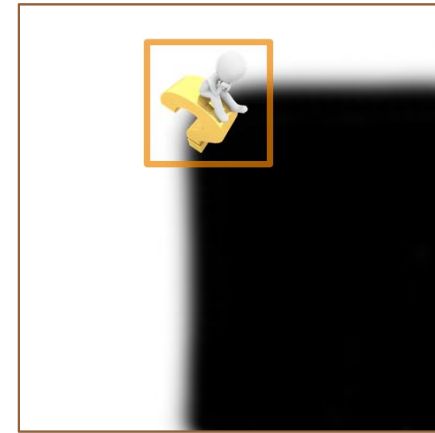
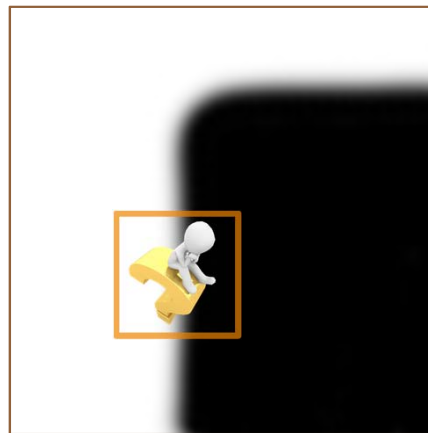
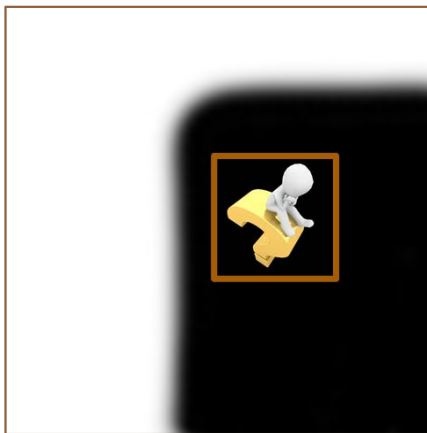


20 most prominent lines

Corner detection

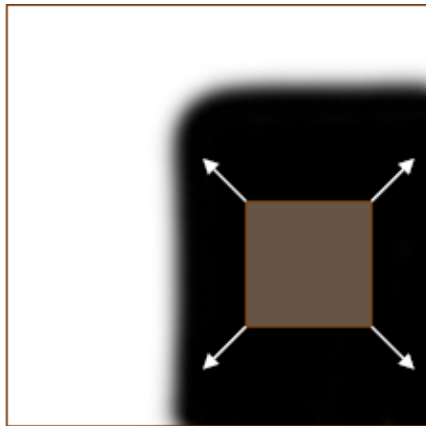
Basic Idea

- Look for image regions that are unusual
 - Lead to unambiguous matches in other images
- Suppose we only consider a small window of pixels
 - What defines whether a feature is a good or bad candidate?

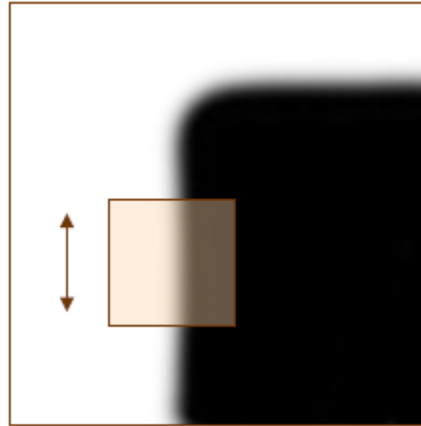


Basic Idea

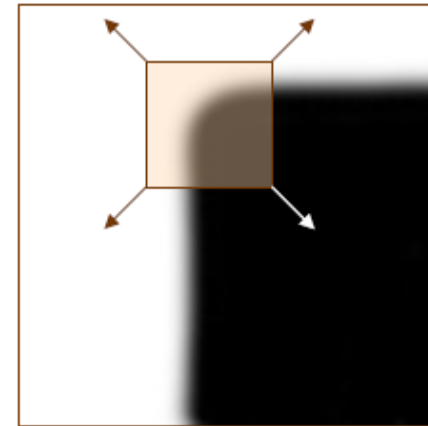
- We should easily recognize the point by looking through a small window
- Shifting a window in any direction should give a large change in intensity



“flat”
no change in
all directions



“edge”
no change along
the edge direction



“corner”
significant change
in all directions

Basic Idea

- Finding Corners

1. Compute the gradient at each point in the image
2. Create the H matrix from the entries in the gradient
3. Compute eigenvectors and eigenvalues
4. Use threshold on eigenvalues to detect corners

Harris corner detection

- Consider shifting the window w by (u, v)
 - Change of intensity for the shift (u, v)

$$E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

Diagram illustrating the components of the Harris corner detection formula:

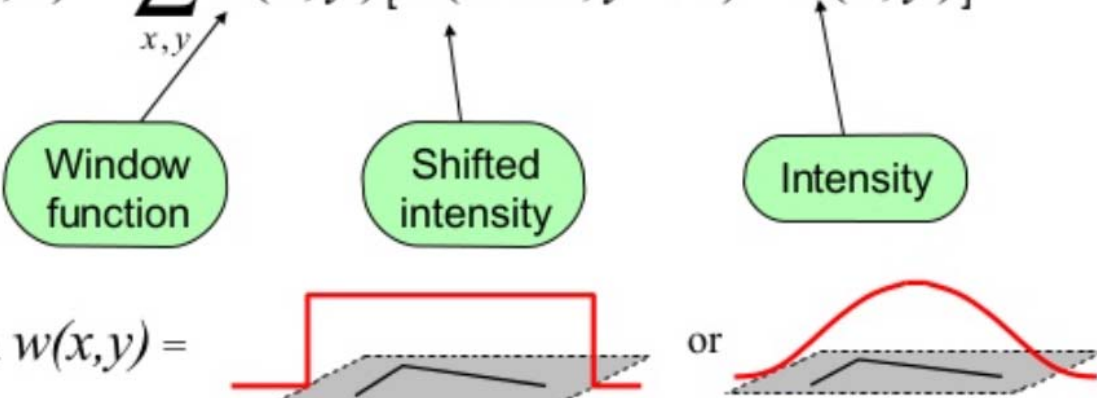
- Window function**: $w(x, y)$
- Shifted intensity**: $I(x+u, y+v)$
- Intensity**: $I(x, y)$

Window function $w(x, y) =$

1 in window, 0 outside

or

Gaussian

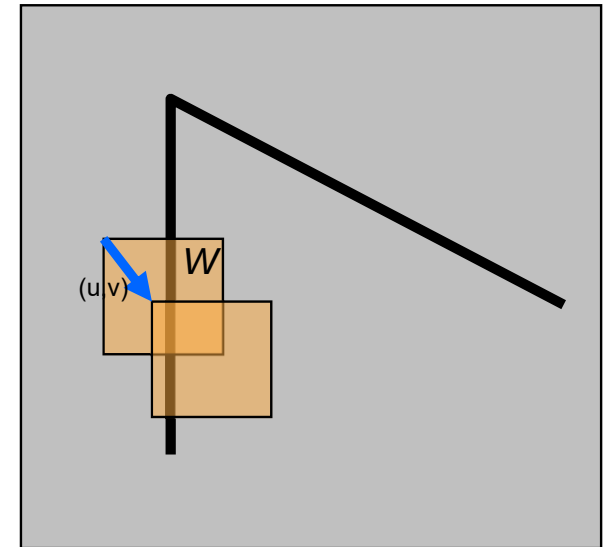


Harris corner detection

- Consider shifting the window W by (u, v)
 - how do the pixels in W change?
 - compare each pixel before and after by summing up the squared differences (SSD)
 - this defines an SSD “error” $E(u, v)$:

$$E(u, v) = \sum_{(x, y) \in W} [I(x + u, y + v) - I(x, y)]^2$$

- We are happy if this error is high
- Slow to compute exactly for each pixel and each offset (u, v)



Harris corner detection

- Small motion assumption
 - Taylor Series expansion of I :

$$I(x+u, y+v) = I(x, y) + \frac{\partial I}{\partial x}u + \frac{\partial I}{\partial y}v + \text{higher order terms}$$

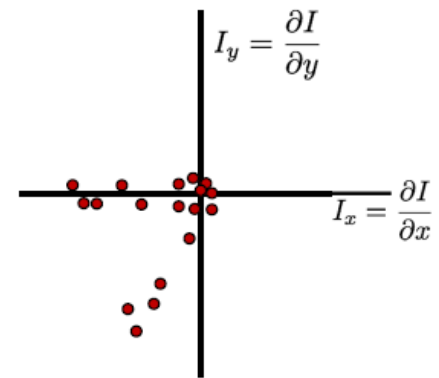
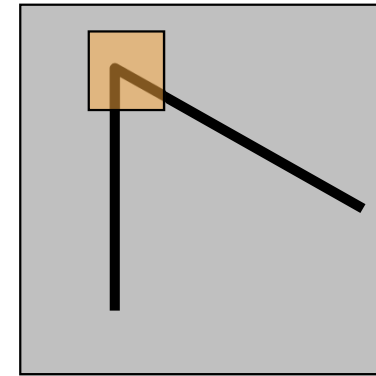
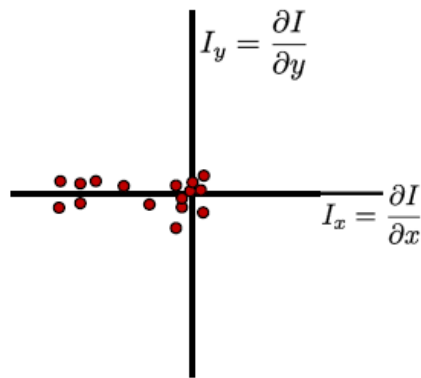
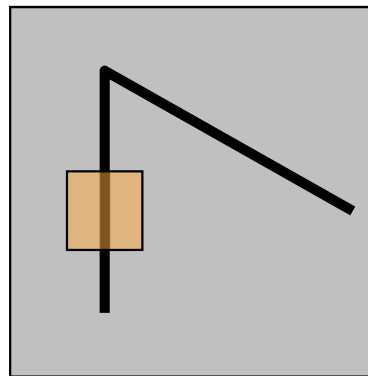
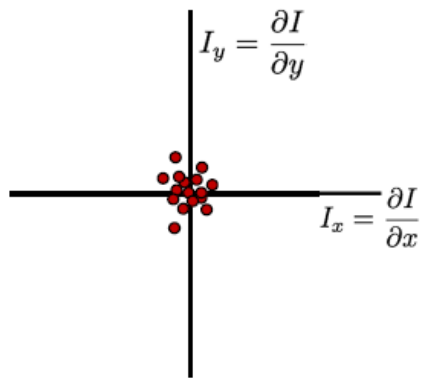
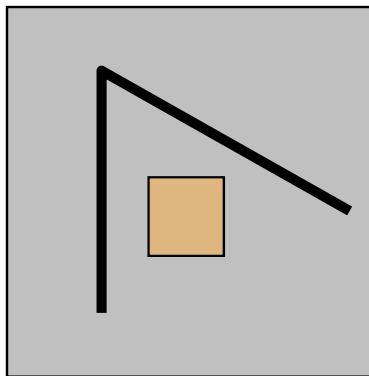
- If the motion (u, v) is small, then first order approximation is good

$$I(x + u, y + v) \approx I(x, y) + \frac{\partial I}{\partial x}u + \frac{\partial I}{\partial y}v \approx I(x, y) + [I_x \ I_y] \begin{bmatrix} u \\ v \end{bmatrix}$$

x gradients shorthand : $I_x = \frac{\partial I}{\partial x}$

Harris corner detection

1. Compute the gradient at each point in the image



Harris corner detection

2. Create the **H** matrix from the entries in the gradient

- Previously defined an SSD “error” $E(u, v)$:

$$\begin{aligned} E(u, v) &= \sum_{(x,y) \in W} [I(x+u, y+v) - I(x, y)]^2 \\ &\approx \sum_{(x,y) \in W} [I(x, y) + I_x u + I_y v - I(x, y)]^2 \\ &\approx \sum_{(x,y) \in W} [I_x u + I_y v]^2 \\ &\approx Au^2 + 2Buv + Cv^2 \approx \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} A & B \\ B & C \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} \end{aligned}$$
$$A = \sum_{(x,y) \in W} I_x^2$$
$$B = \sum_{(x,y) \in W} I_x I_y$$
$$C = \sum_{(x,y) \in W} I_y^2$$

Thus, $E(u, v)$ is locally approximated as a quadratic error function

Harris corner detection

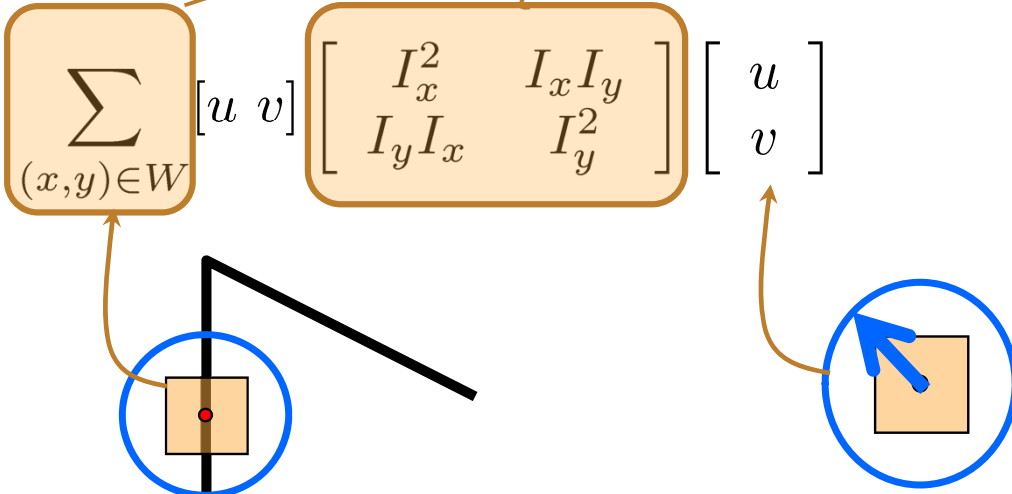
2. Create the **H** matrix from the entries in the gradient

$$E(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} A & B \\ B & C \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

$$H = \sum \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

'second moment' matrix

- This can be rewritten:

$$E(u, v) = \sum_{(x,y) \in W} \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} I_x^2 & I_x I_y \\ I_y I_x & I_y^2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$


Harris corner detection

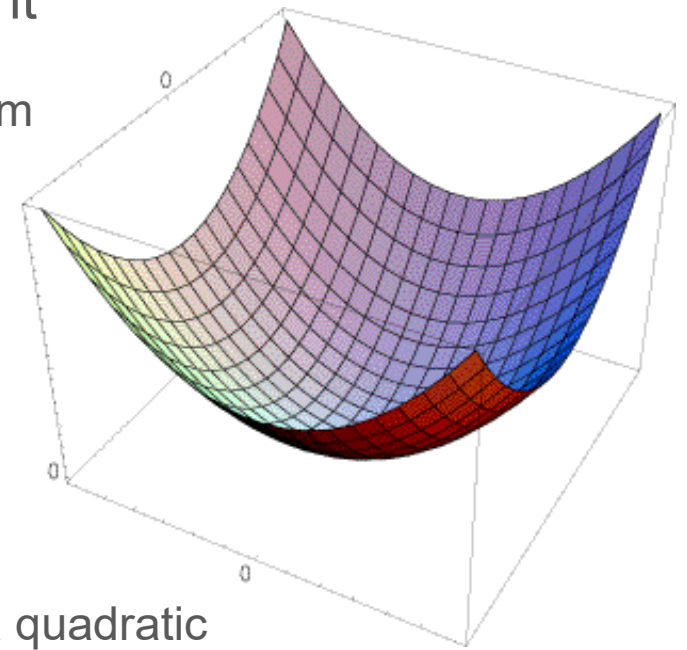
2. Create the **H** matrix from the entries in the gradient

- The surface $E(u,v)$ is locally approximated by a quadratic form

$$E(u,v) \approx [u \ v] H \begin{bmatrix} u \\ v \end{bmatrix}$$

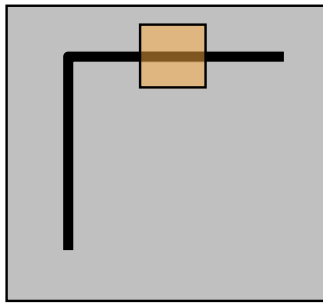
$$H = \sum \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

- By computing the gradient covariance matrix, we are fitting a quadratic to the gradients over a small image region



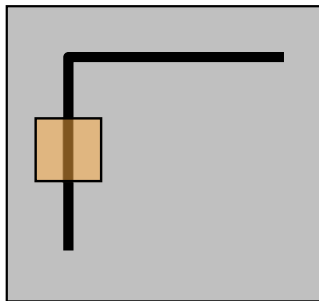
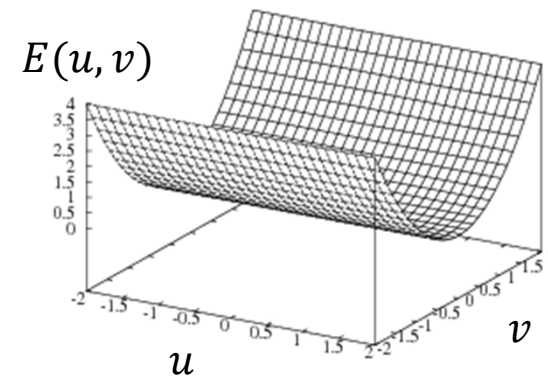
Harris corner detection

2. Create the **H** matrix from the entries in the gradient



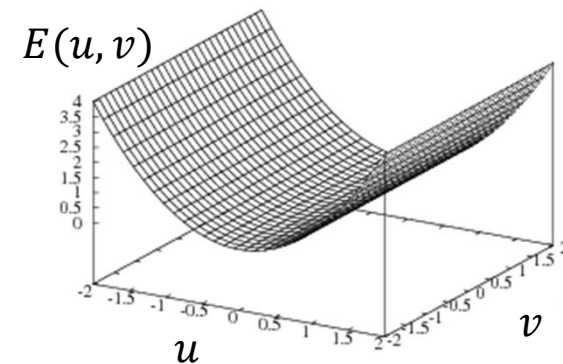
Horizontal edge: $I_x = 0$

$$H = \begin{bmatrix} 0 & 0 \\ 0 & C \end{bmatrix}$$



Vertical edge: $I_y = 0$

$$H = \begin{bmatrix} A & 0 \\ 0 & 0 \end{bmatrix}$$



Harris corner detection

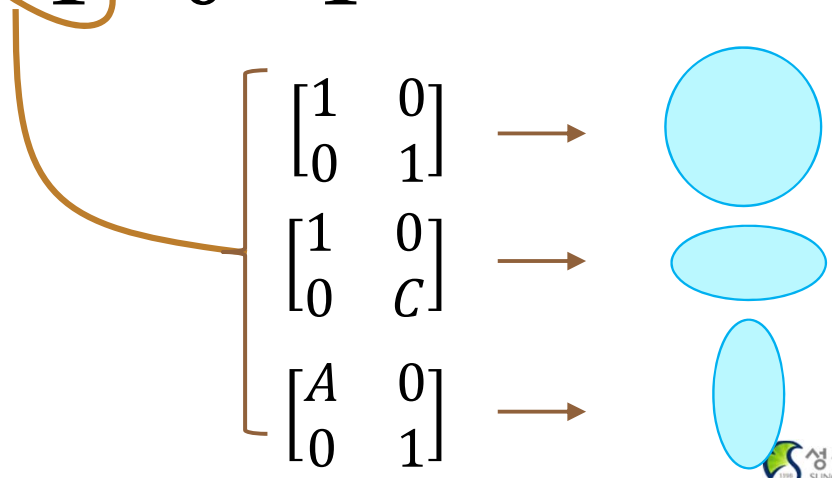
3. Compute eigenvectors and eigenvalues

- Singular Value Decomposition (SVD)

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}^T$$

eigenvectors eigenvalues

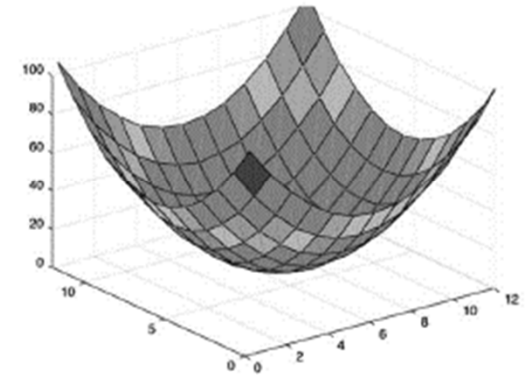
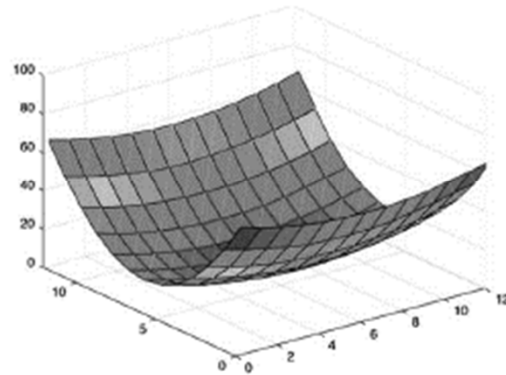
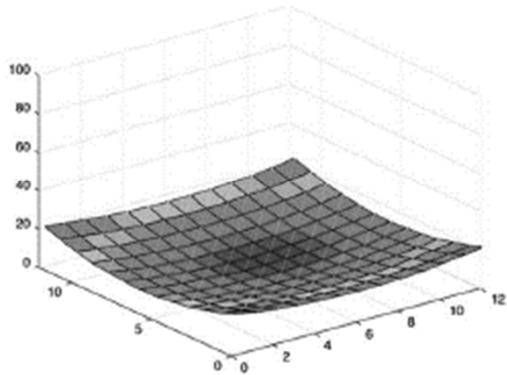
axis of the 'ellipse slice'



Harris corner detection

3. Compute eigenvectors and eigenvalues

- Which error surface indicates a good image feature?



Harris corner detection

3. Compute eigenvectors and eigenvalues

- The eigenvectors of a matrix A are the vectors e that satisfy:

$$Ae = \lambda e$$

- The scalar λ is the eigenvalue corresponding to e
- The **eigenvalues are found** by solving: $\det(A - \lambda I) = 0$
- In our case, $A = H$ is a 2x2 matrix and know λ then we **find eigenvector e** by solving

$$\begin{bmatrix} h_{11} - \lambda & h_{12} \\ h_{21} & h_{22} - \lambda \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0$$

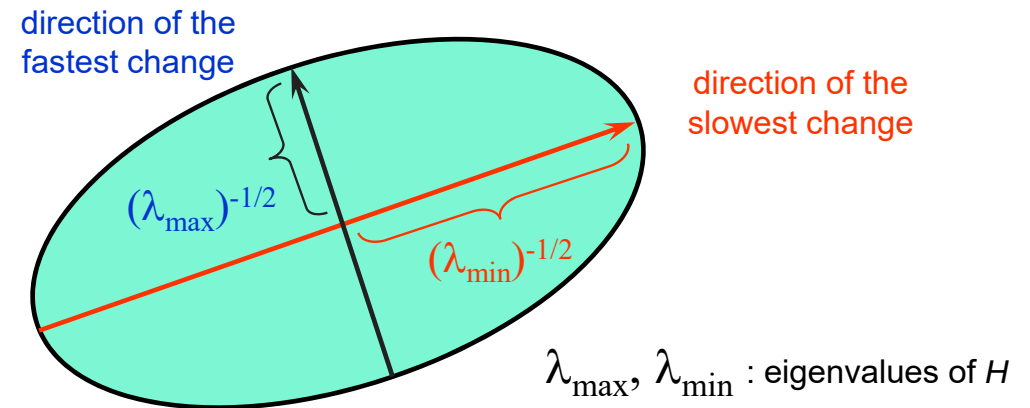
Harris corner detection

3. Compute eigenvectors and eigenvalues

- Since H is symmetric, we have
$$H = R^{-1} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} R$$
- We can visualize H as an ellipse with axis lengths determined by the *eigenvalues* of H and orientation determined by the *eigenvectors* R of H

Ellipse equation:

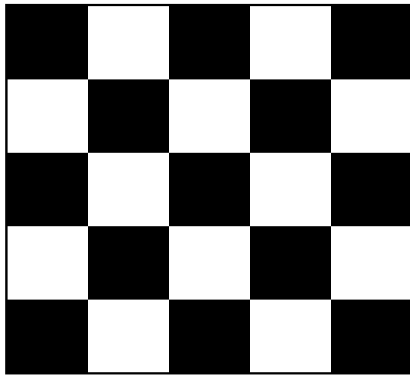
$$\begin{bmatrix} u & v \end{bmatrix} H \begin{bmatrix} u \\ v \end{bmatrix} = \text{const}$$



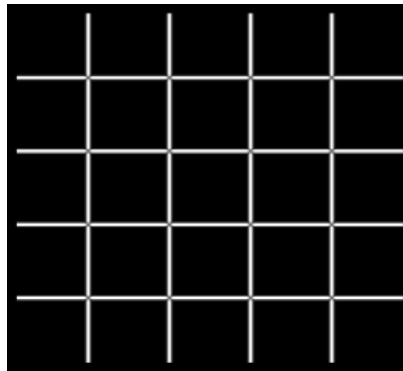
Harris corner detection

3. Compute eigenvectors and eigenvalues

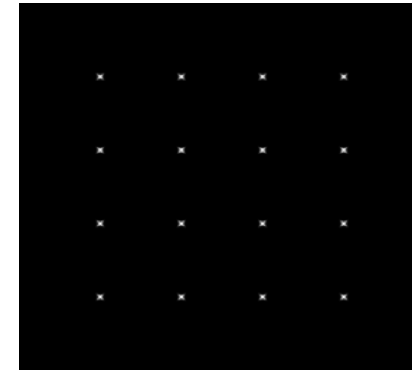
- How are $\lambda_{\max}$ and $\lambda_{\min}$ relevant for feature detection?
 - What's our feature scoring function?
- Want $E(u,v)$ to be large for small shifts in all directions
 - the minimum of $E(u,v)$ should be large, over all unit vectors $[u \ v]$
 - this minimum is given by the smaller eigenvalue ($\lambda_{\min}$) of H



I



$\lambda_{\max}$

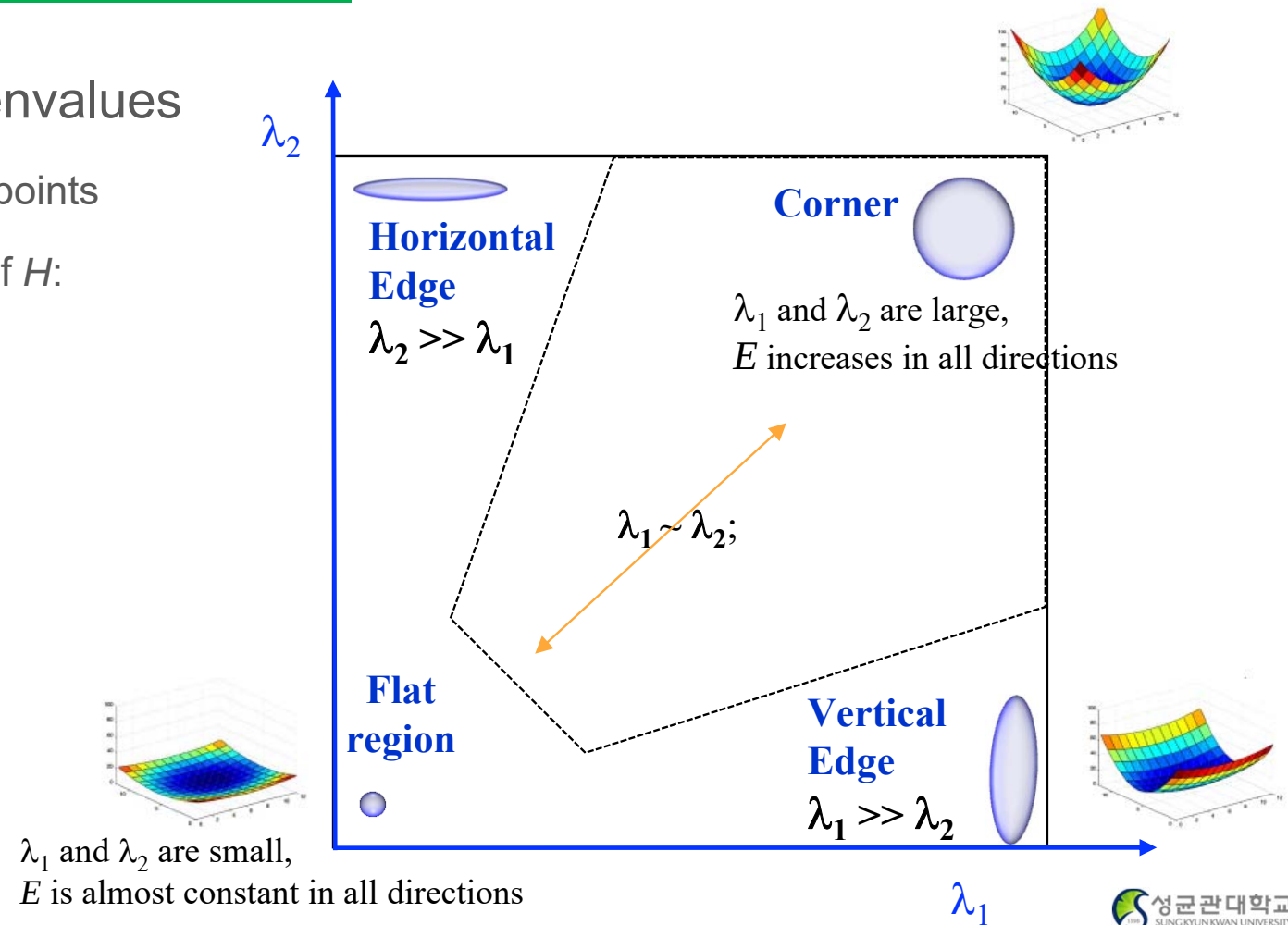


$\lambda_{\min}$

Harris corner detection

■ Interpreting the eigenvalues

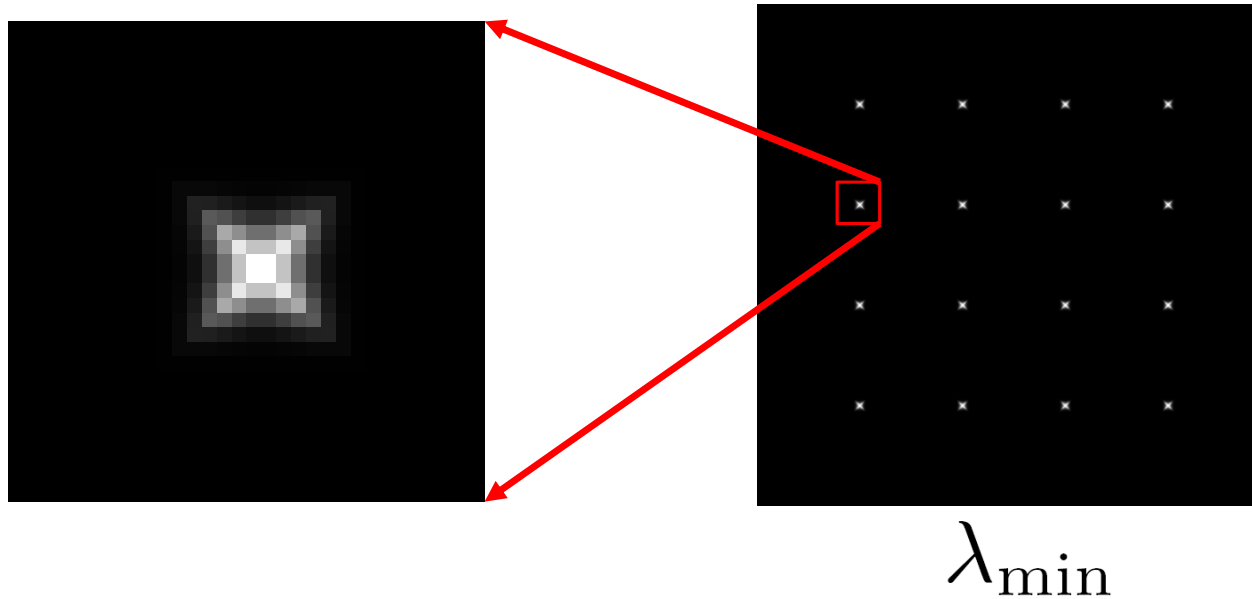
- Classification of image points using eigenvalues of H :



Harris corner detection

4. Use threshold on eigenvalues to detect corners

- Find points with large response ($\lambda_{\min} > \text{threshold}$)
- Choose those points where $\lambda_{\min}$ is a local maximum as features



Harris corner detection

4. Use threshold on eigenvalues to detect corners

- Use the smallest eigenvalue as the response function

- Harris & Stephens (1988)

$$R = \det(H) - \alpha \text{trace}(H)^2$$

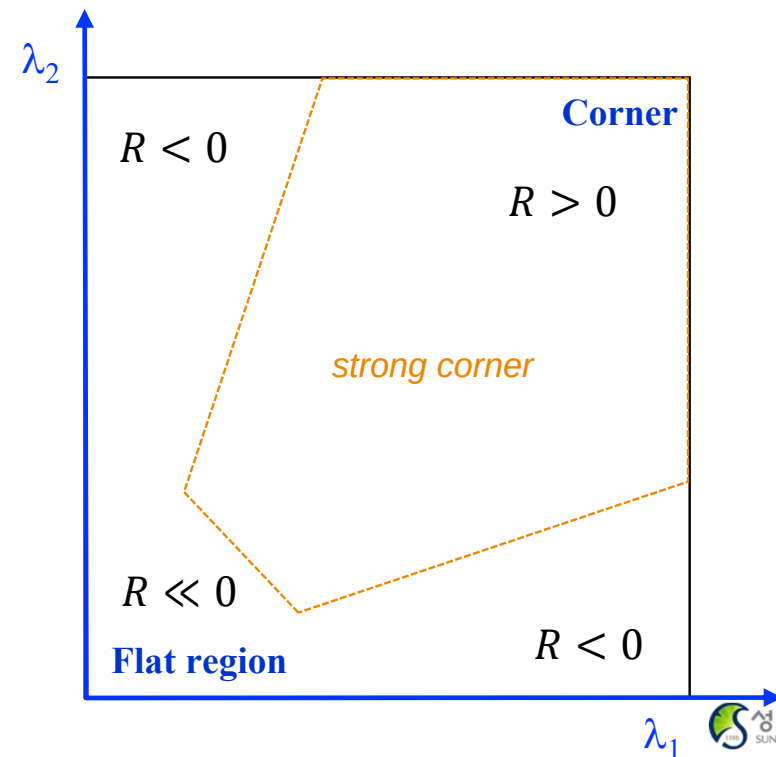
- Shi & Tomasi (1994)

$$R = \min(\lambda_1, \lambda_2)$$

- Nobel (1998)

$$R = \frac{\det(H)}{\text{trace}(H) + \varepsilon}$$

https://docs.opencv.org/master/d4/d8c/tutorial_py_shi_tomasi.html

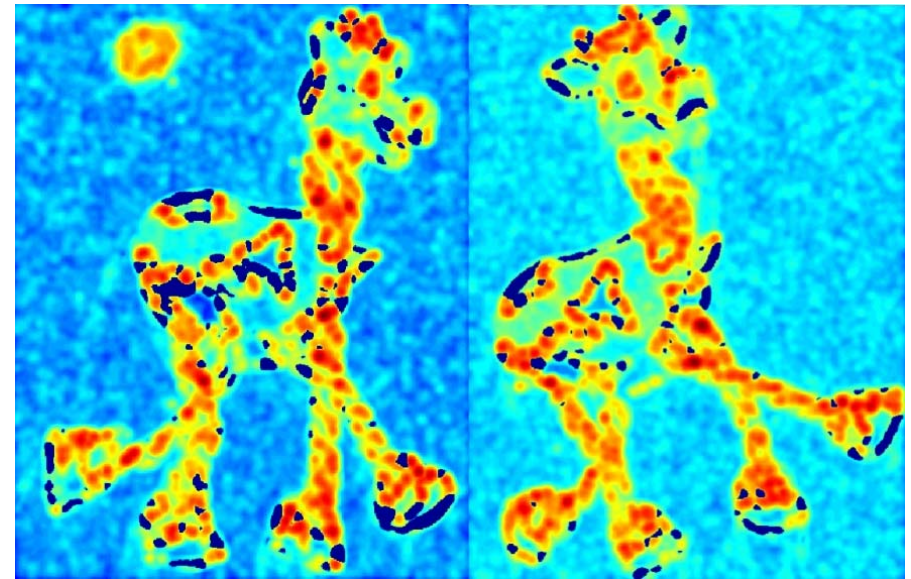


Harris corner detection

- Harris detector example (steps)



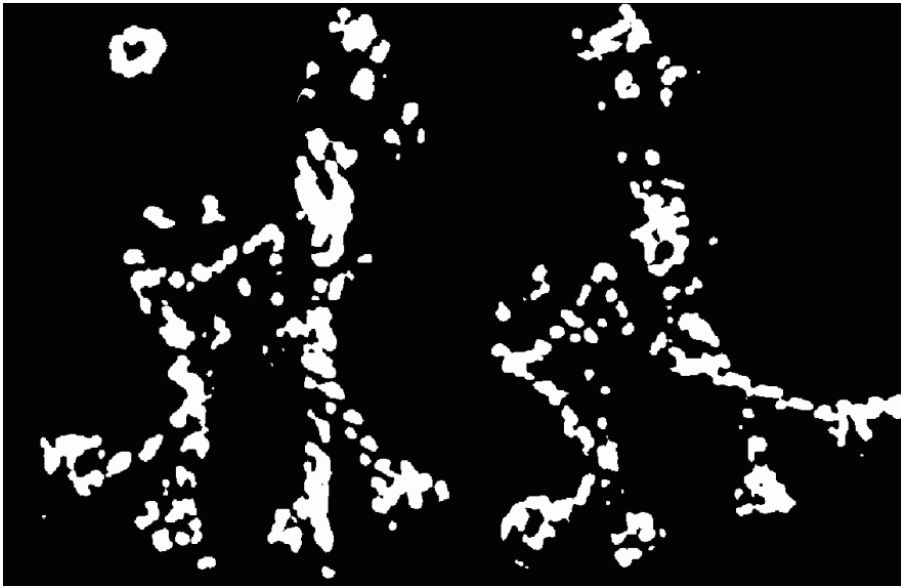
Input



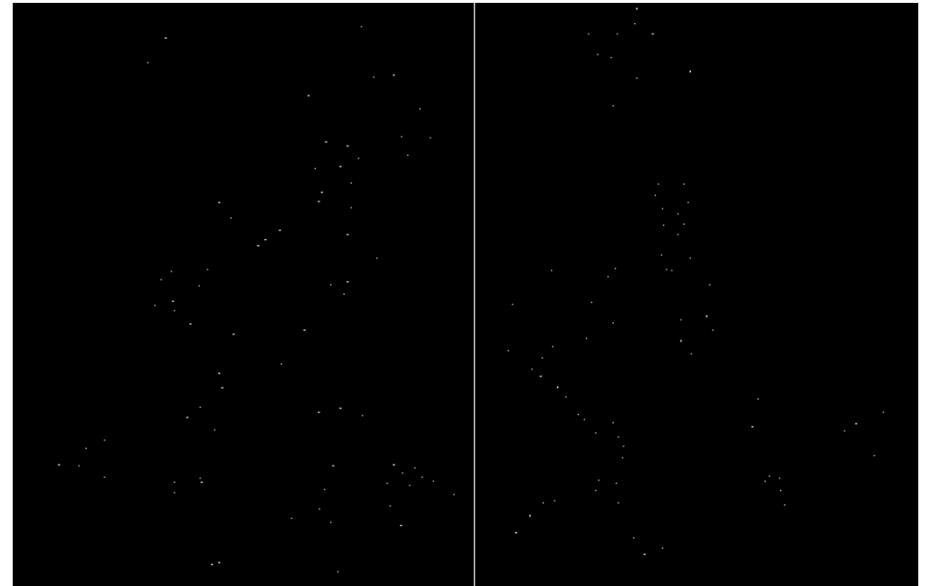
Corner response (red high, blue low)

Harris corner detection

- Harris detector example (steps)



Find points with large corner response: $R > \text{threshold}$



Find local maxima of R

Harris corner detection

- Harris detector example (steps)



Result : Harris features (in red)

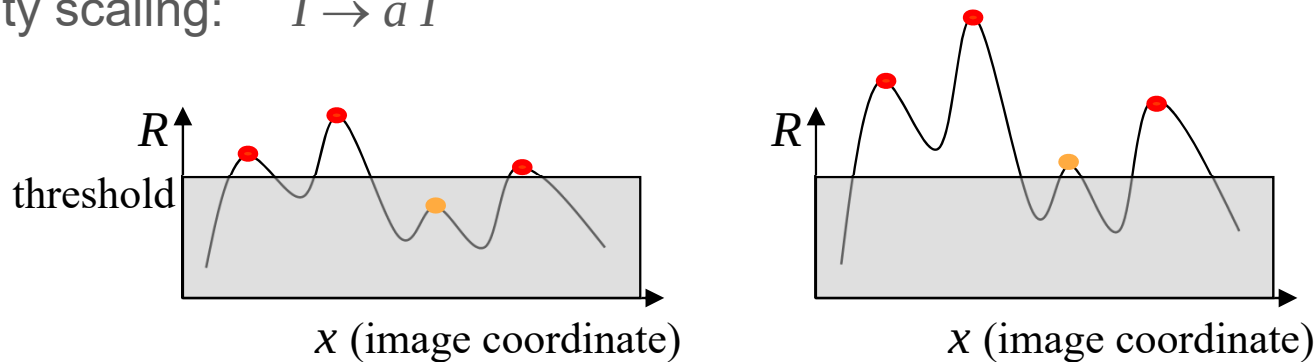
https://docs.opencv.org/master/dc/d0d/tutorial_py_features_harris.html

Invariance properties : harris detector

- Affine intensity change



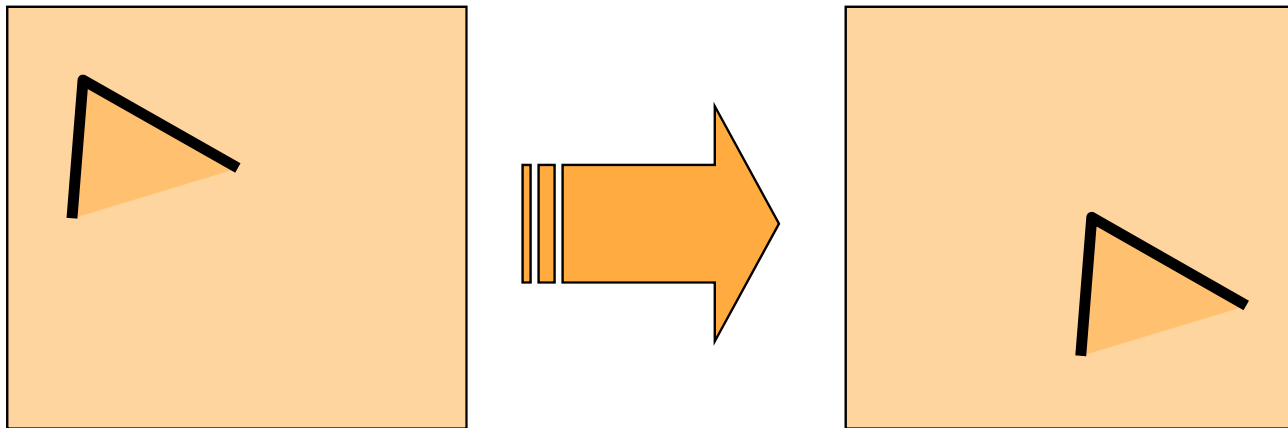
- Only derivatives are used $\Rightarrow$ invariance to intensity shift $I \rightarrow I + b$
- Intensity scaling: $I \rightarrow aI$



Partially invariant to affine intensity change

Invariance properties : harris detector

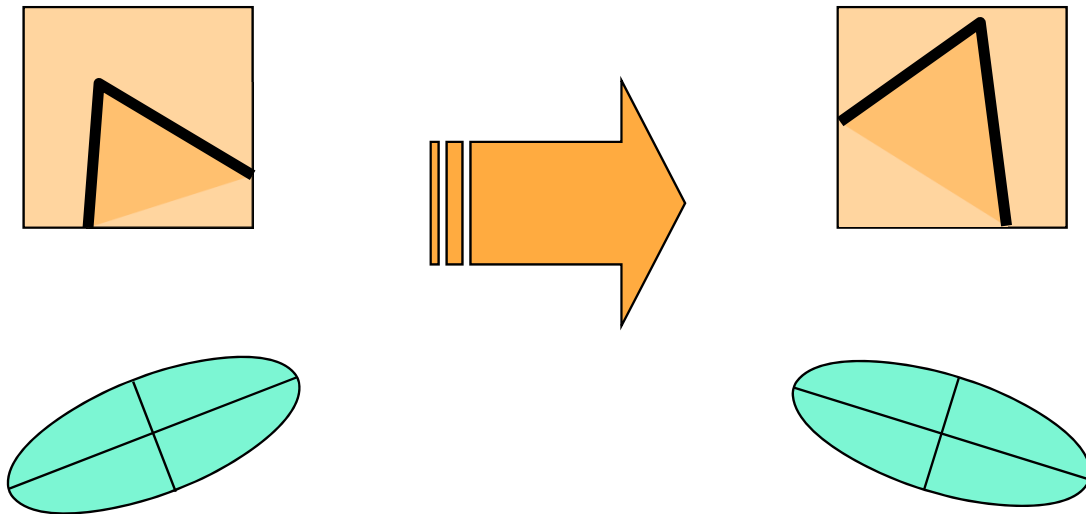
- Image translation



Corner location is covariant w.r.t. translation

Invariance properties : harris detector

- Image rotation

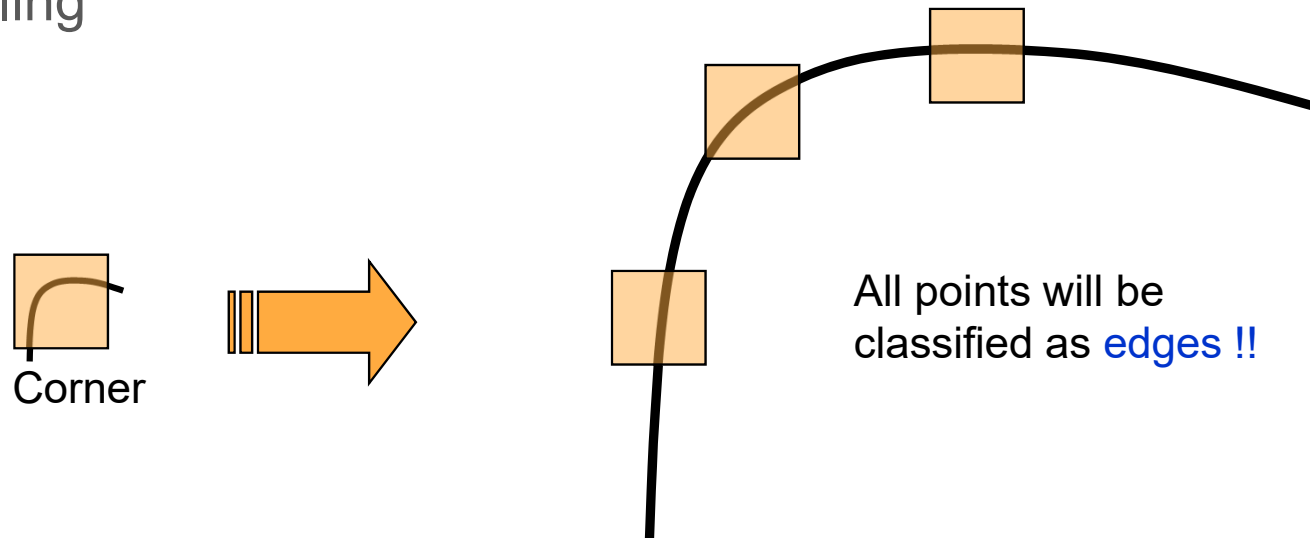


Second moment ellipse rotates but its shape (i.e. eigenvalues) remains the same

Corner location is covariant w.r.t. image rotation

Invariance properties : harris detector

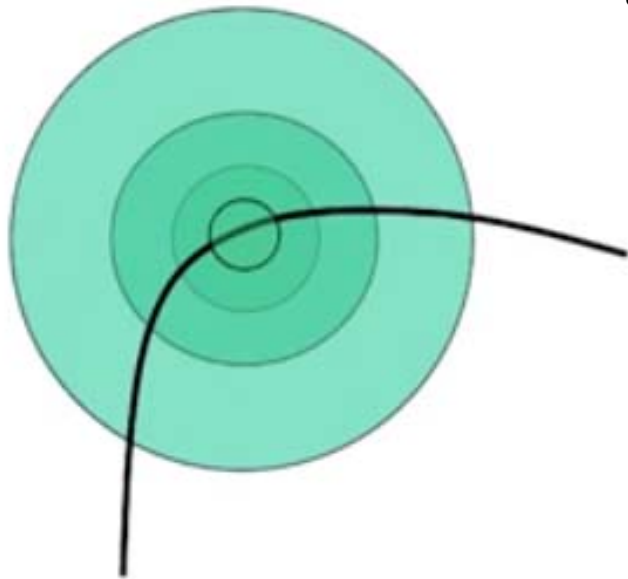
- Image scaling



Not invariant to scaling

Scale invariant detection

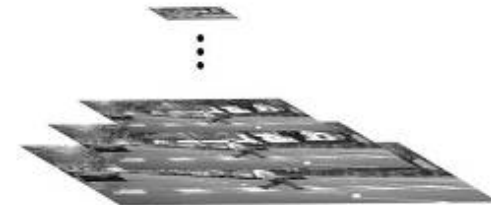
Suppose you're looking for corners



- Key idea ??: find scale that gives local maximum of f
✓ in both position and scale



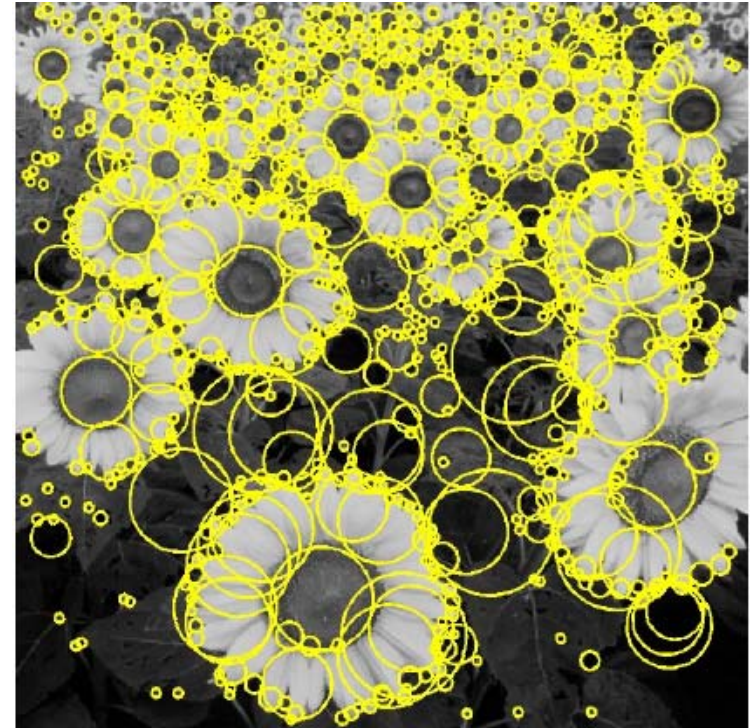
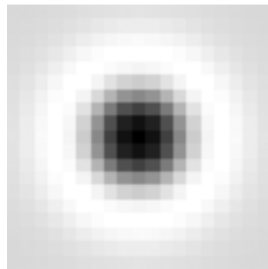
- Instead of computing f for larger and larger windows, we can implement using a fixed window size with a Gaussian pyramid



Blob detection

Basic Idea

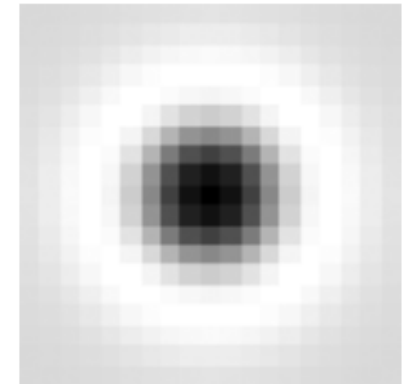
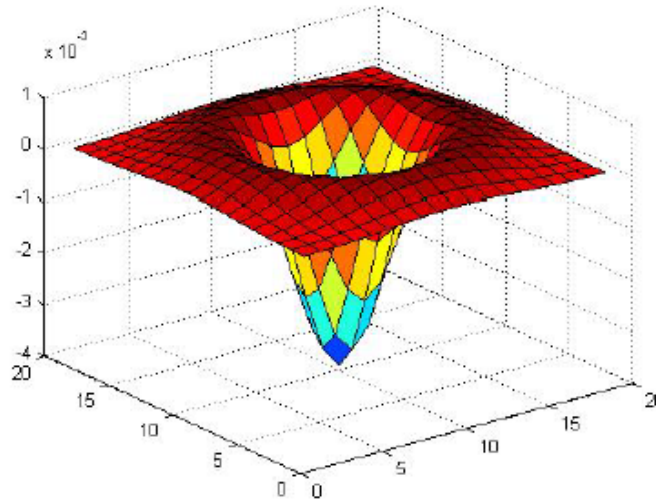
- How can an edge finder be used to find blobs in images ?
 - Convolve the image with a blob filter at multiple scales and look for extrema of filter response in the resulting scale space.
- Laplacian of Gaussian looks bit like blob



Blob detection

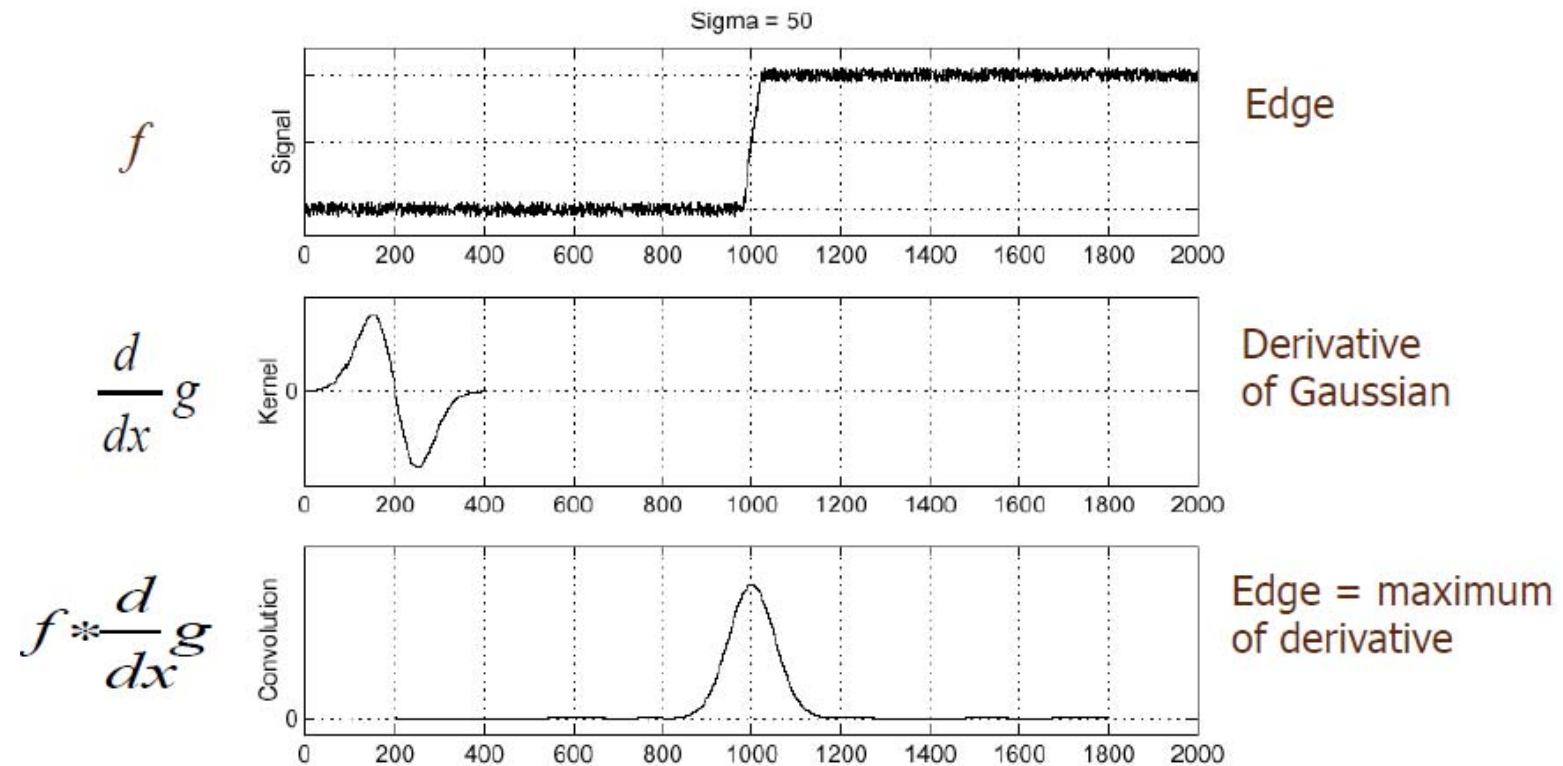
- Laplacian of Gaussian
 - Circularly symmetric operator for blob detection in 2D

$$\nabla^2 g = \frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2}$$



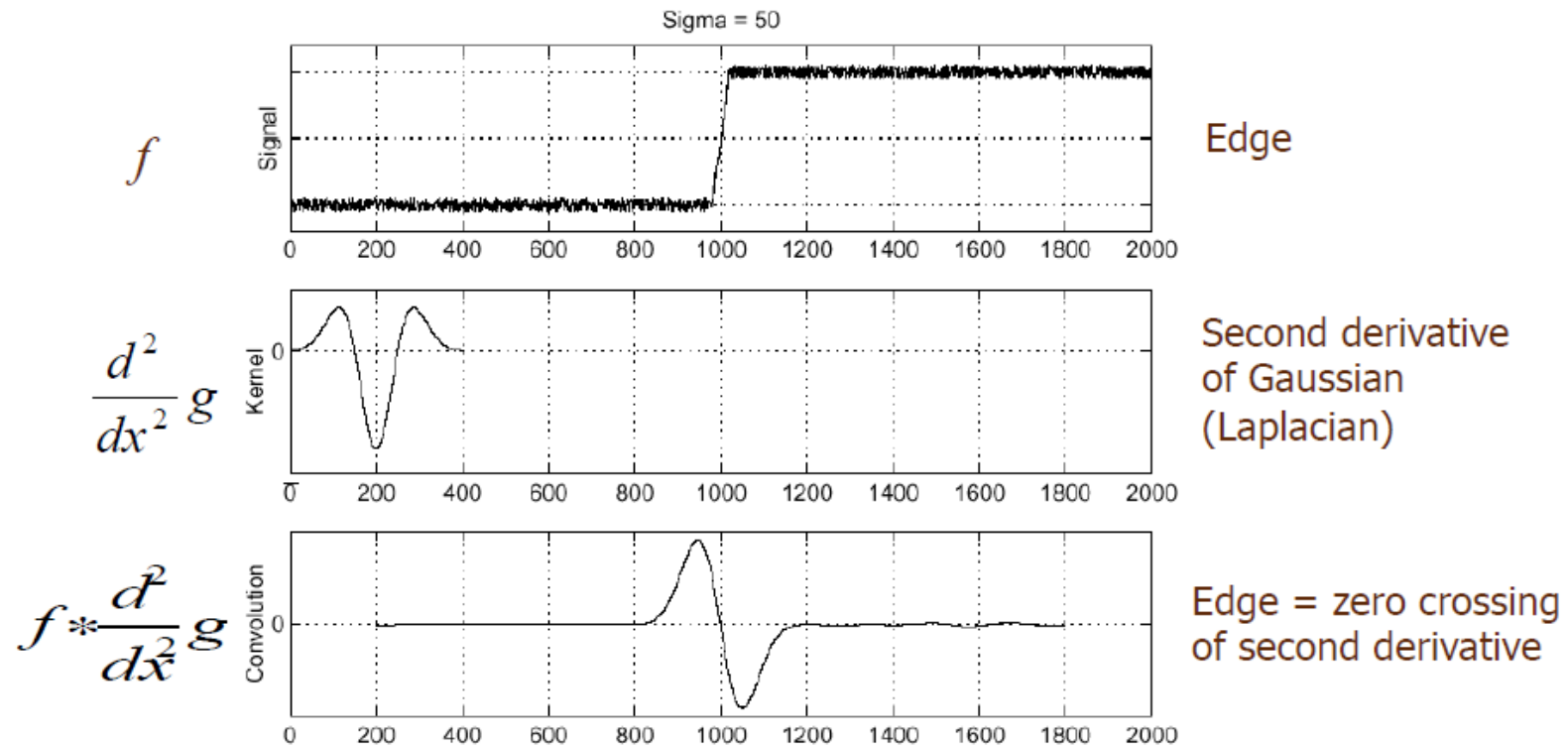
Blob detection

- Laplacian of Gaussian



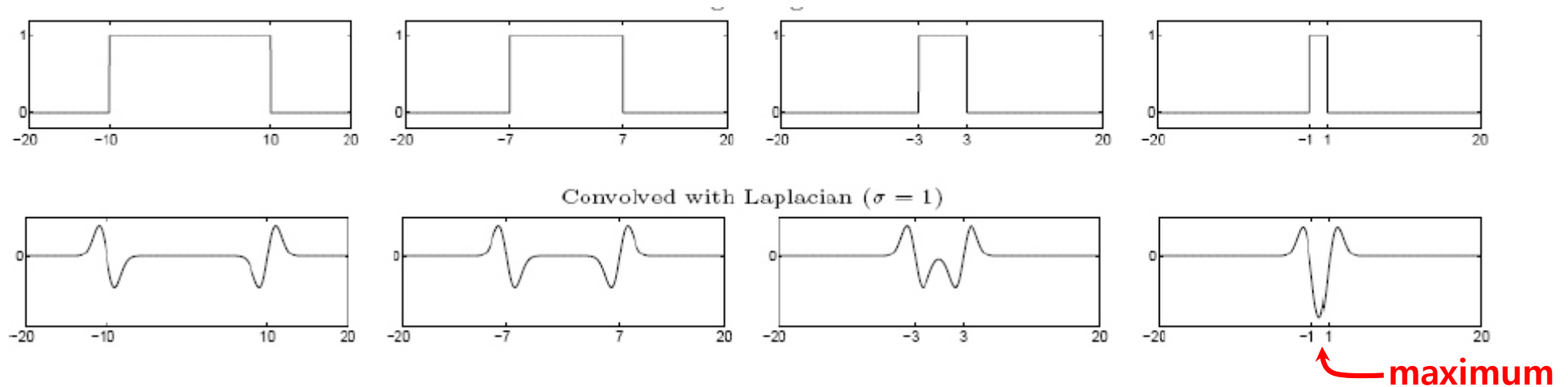
Blob detection

- Laplacian of Gaussian



Blob detection

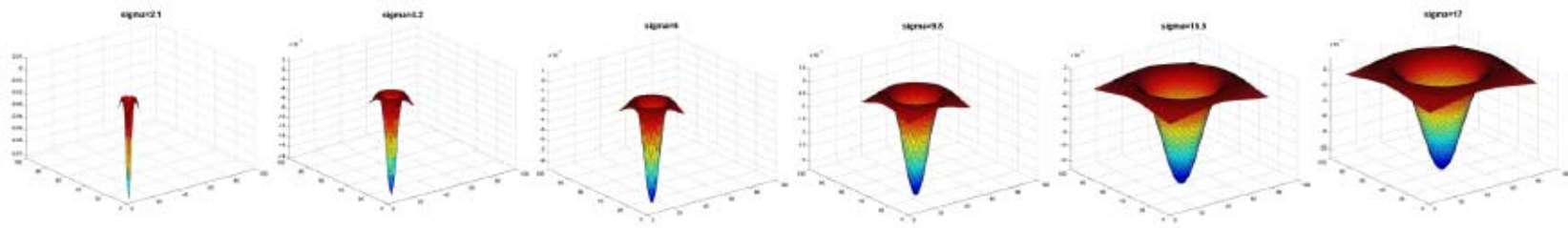
- From edges to blobs
 - Edge = ripple
 - Blob = superposition of two ripples



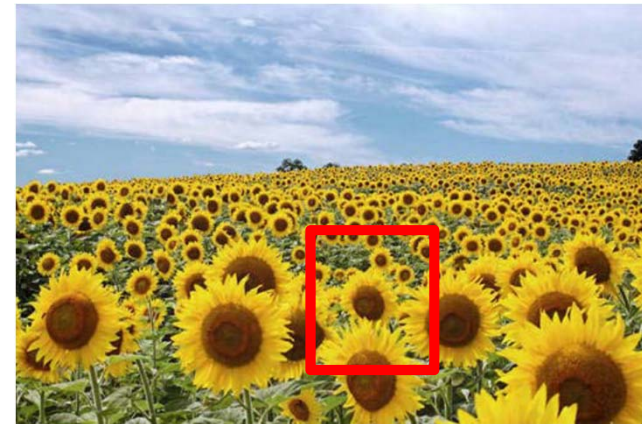
- **Spatial selection:** the magnitude of the Laplacian response will achieve a maximum at the center of the blob, provided the scale of the Laplacian is “matched” to the scale of the blob

Blob detection

- Example



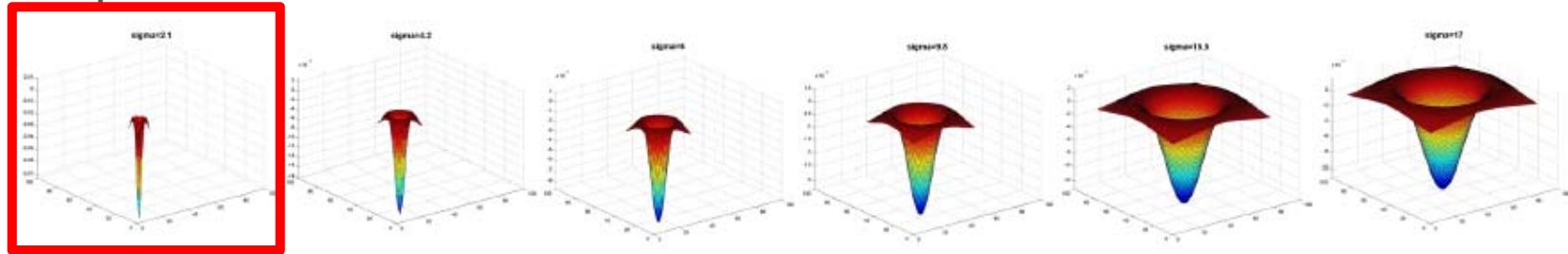
Full size



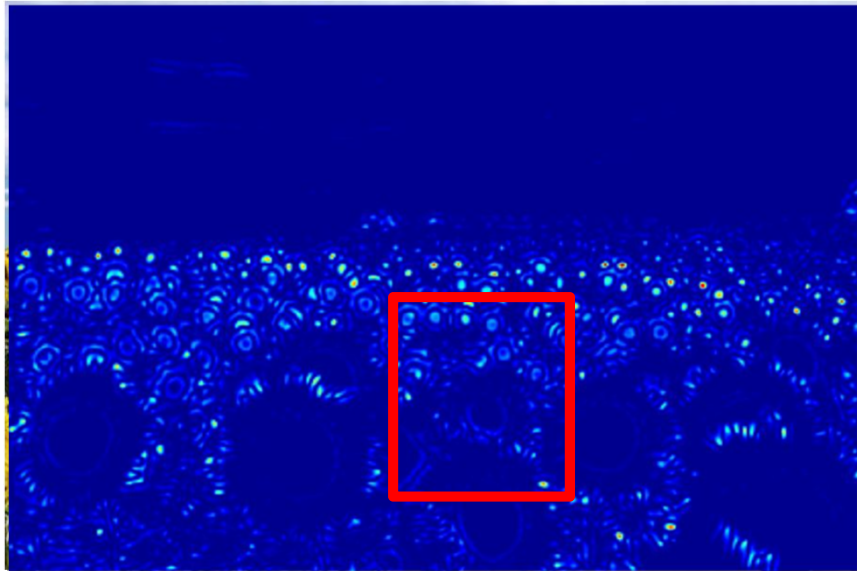
3/4 size

Blob detection

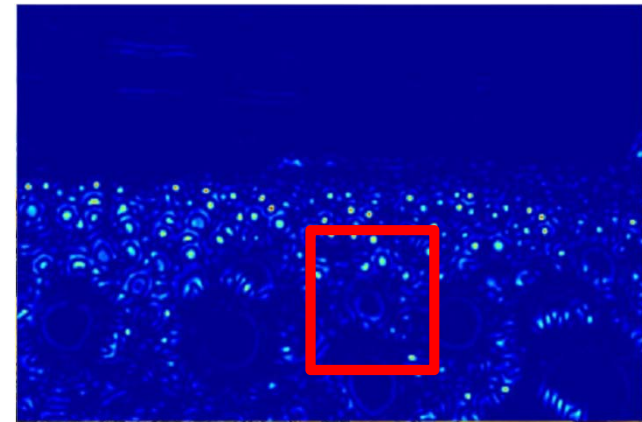
- Example



$\sigma = 2.1$



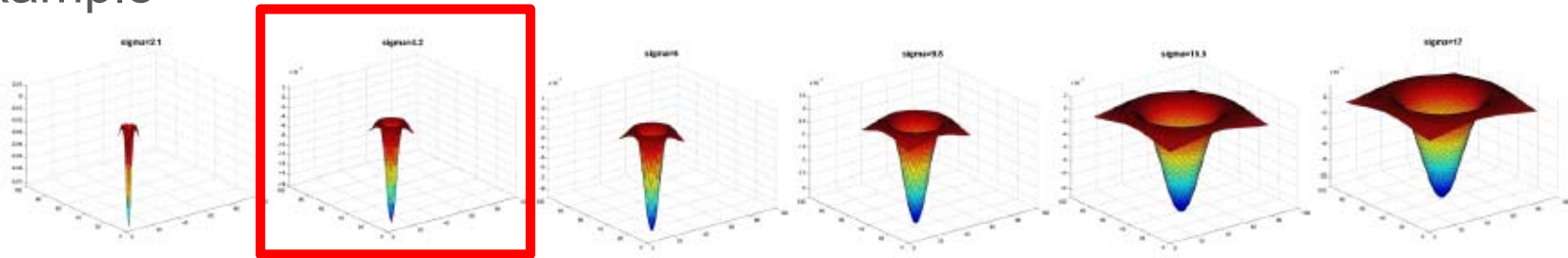
Full size



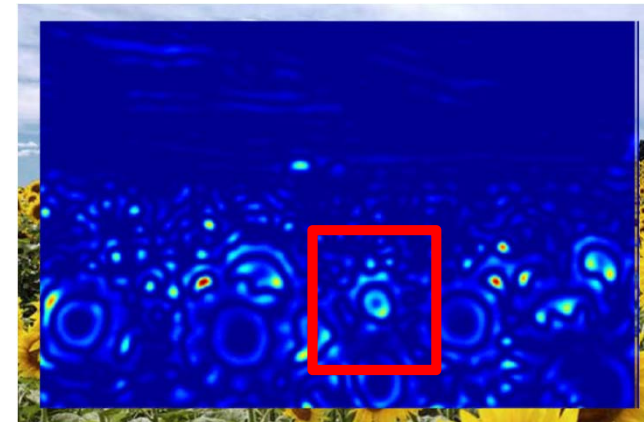
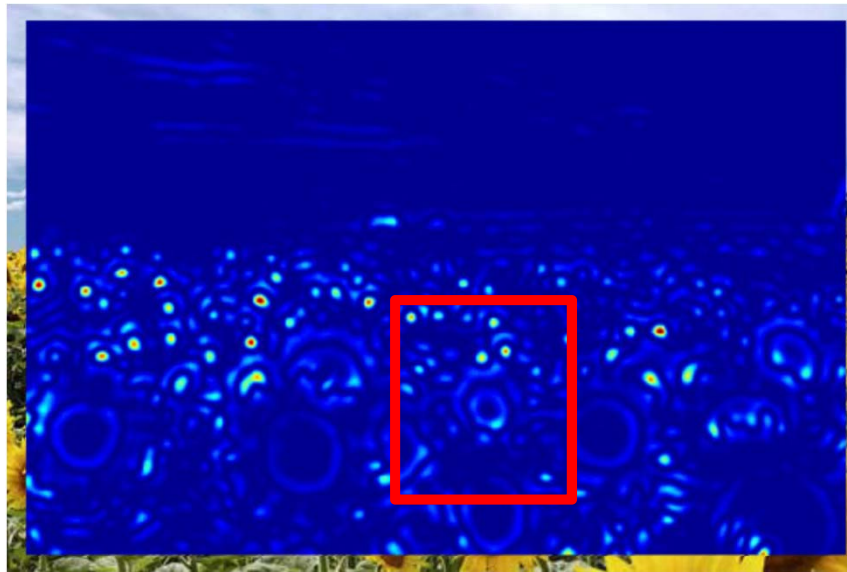
3/4 size

Blob detection

- Example



$\sigma = 4.2$

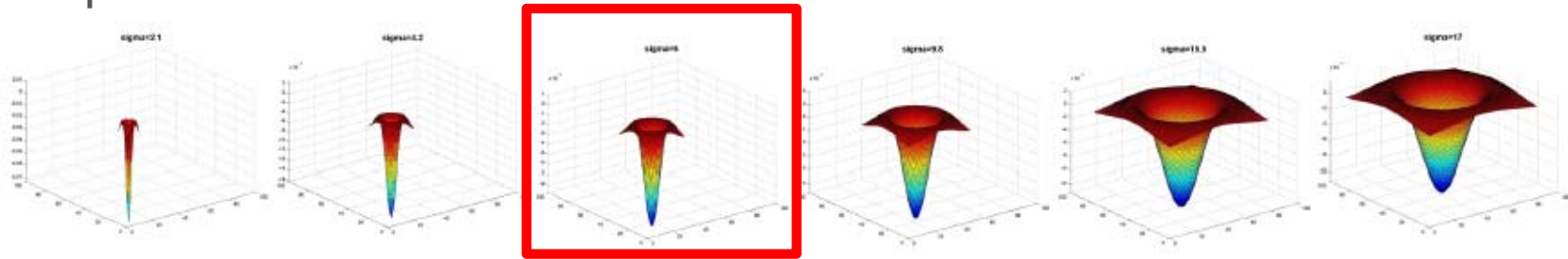


3/4 size

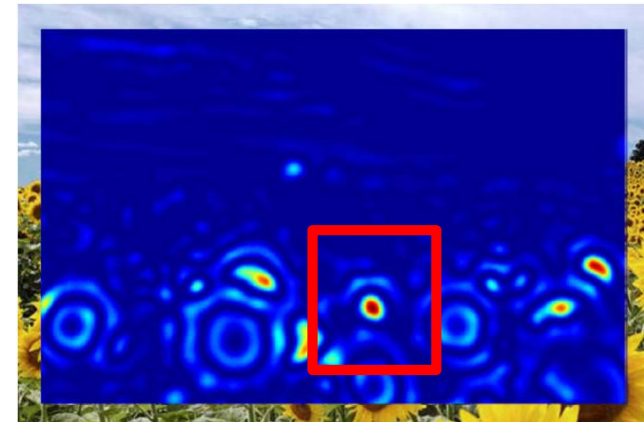
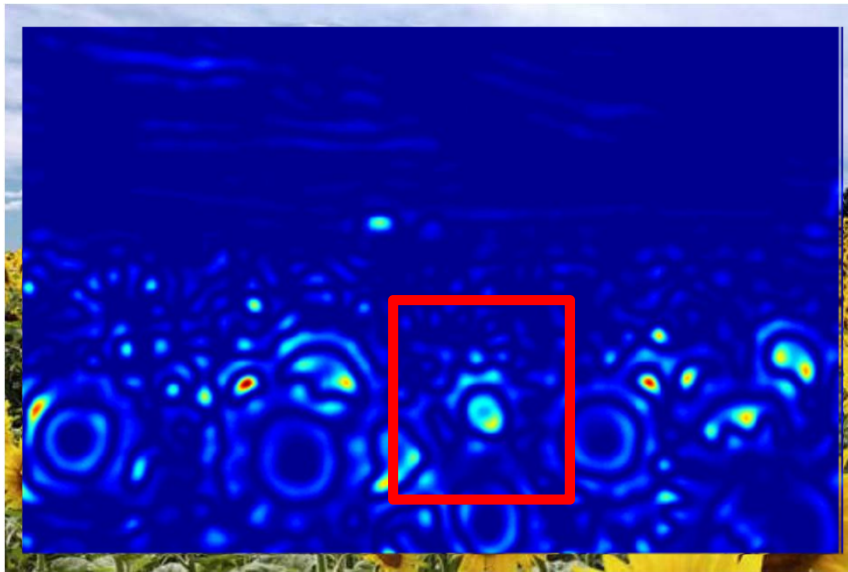
Full size

Blob detection

- Example



$\sigma = 6$



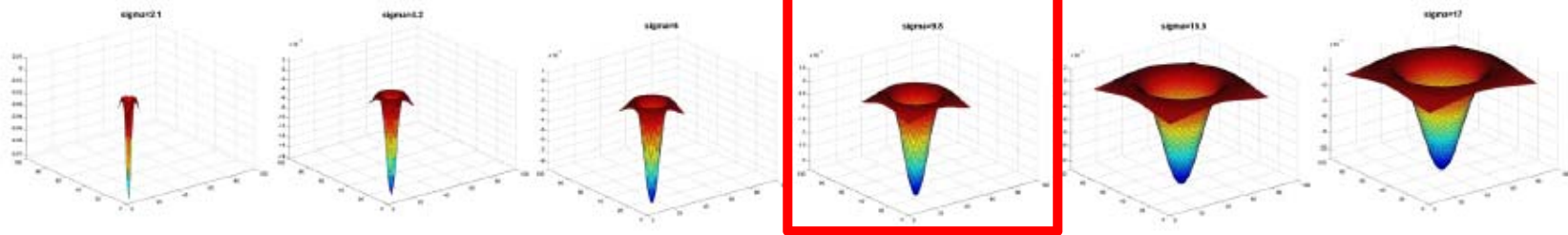
Full size

peak!

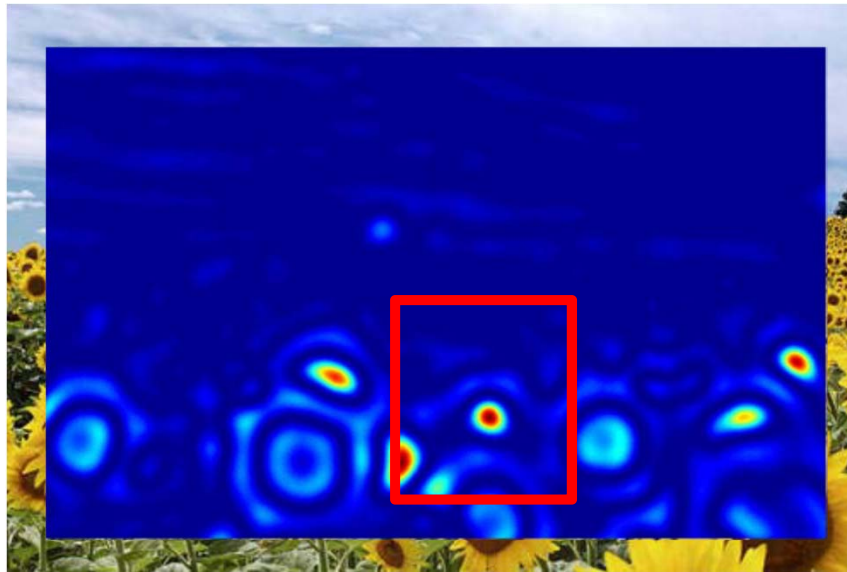
3/4 size

Blob detection

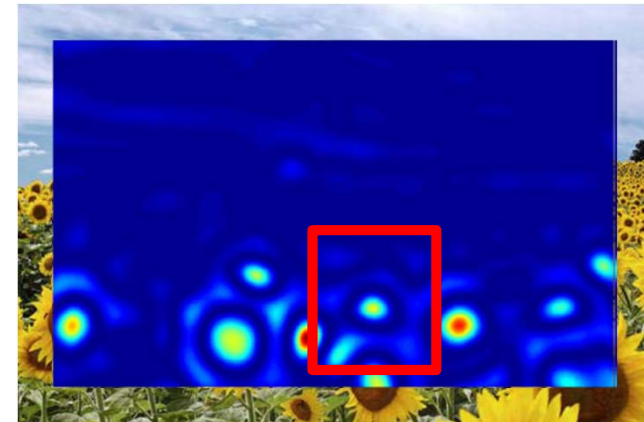
- Example



$\sigma = 9.8$



peak!

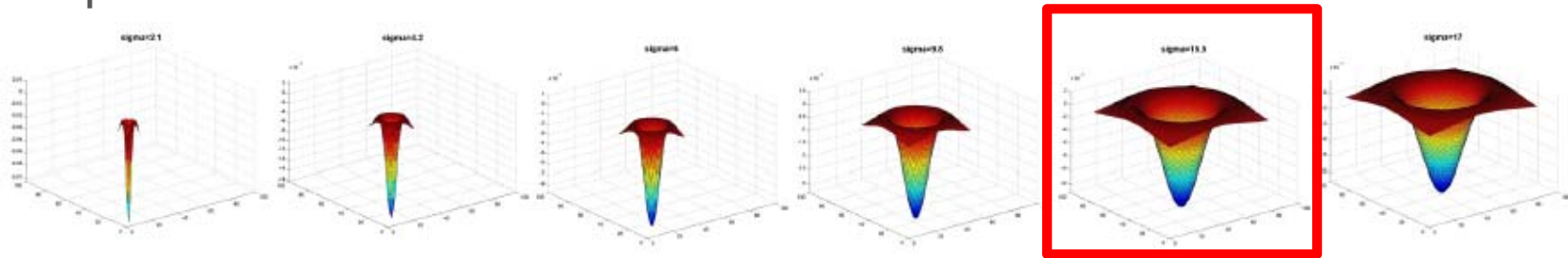


3/4 size

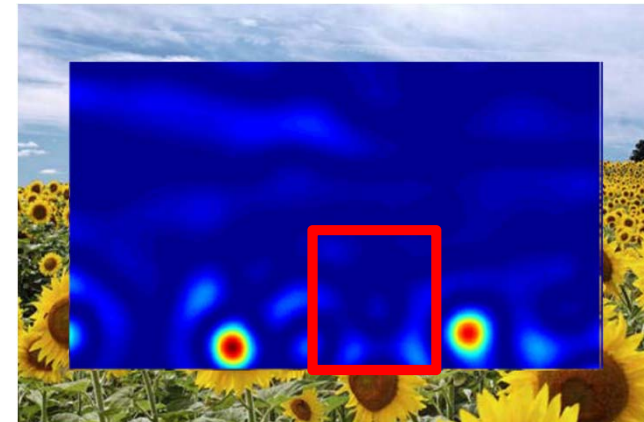
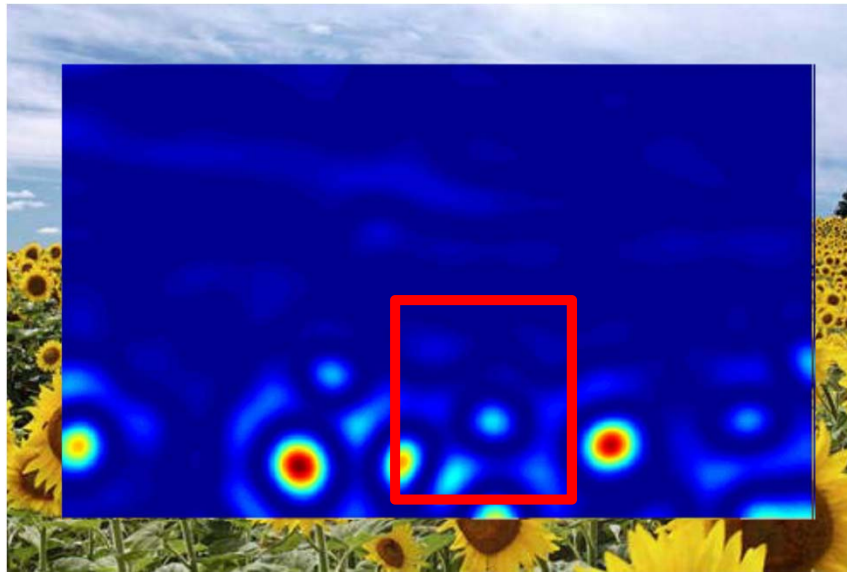
Full size

Blob detection

- Example



$\sigma = 15.5$

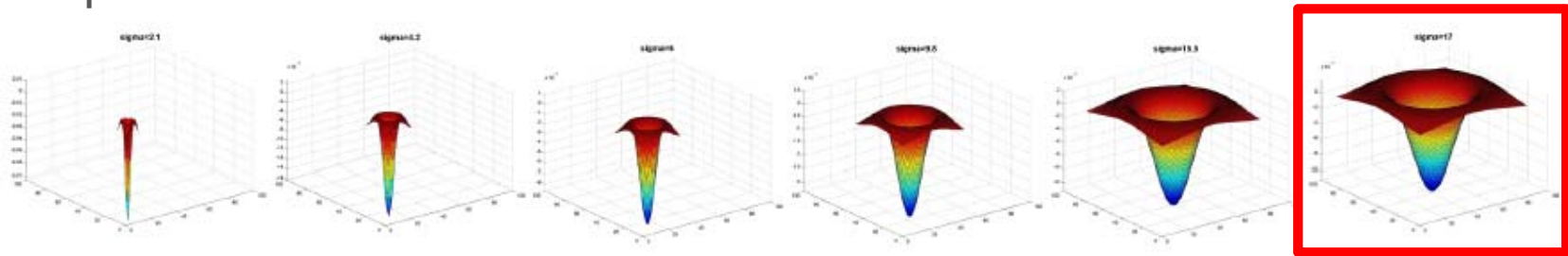


3/4 size

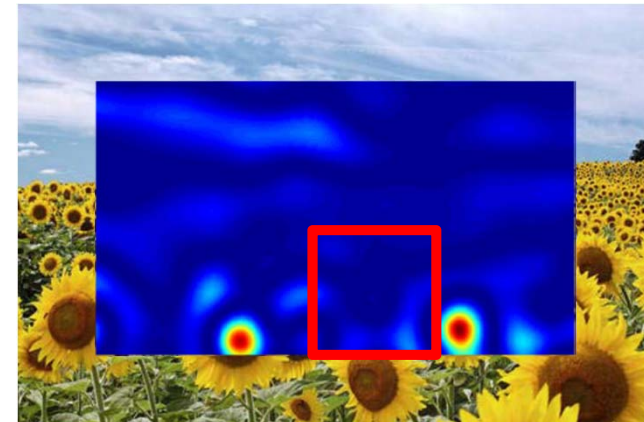
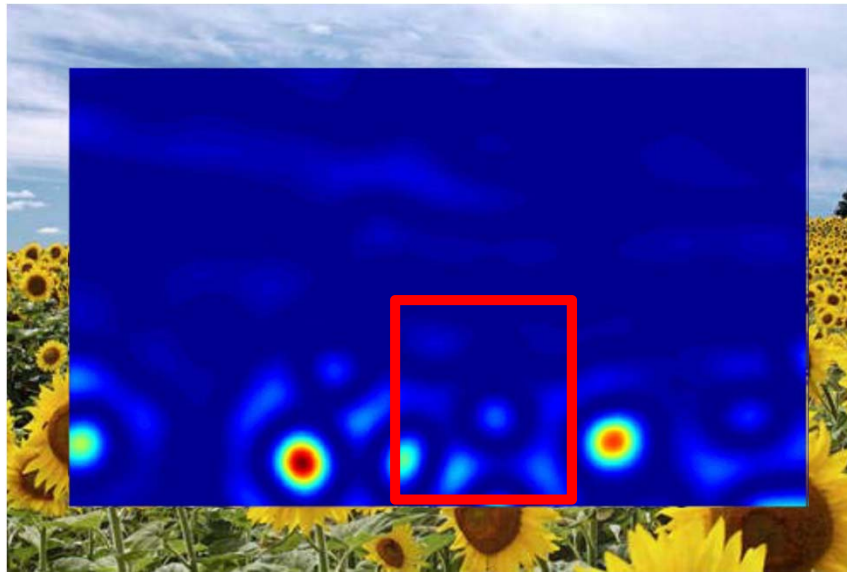
Full size

Blob detection

- Example



$\sigma = 17$

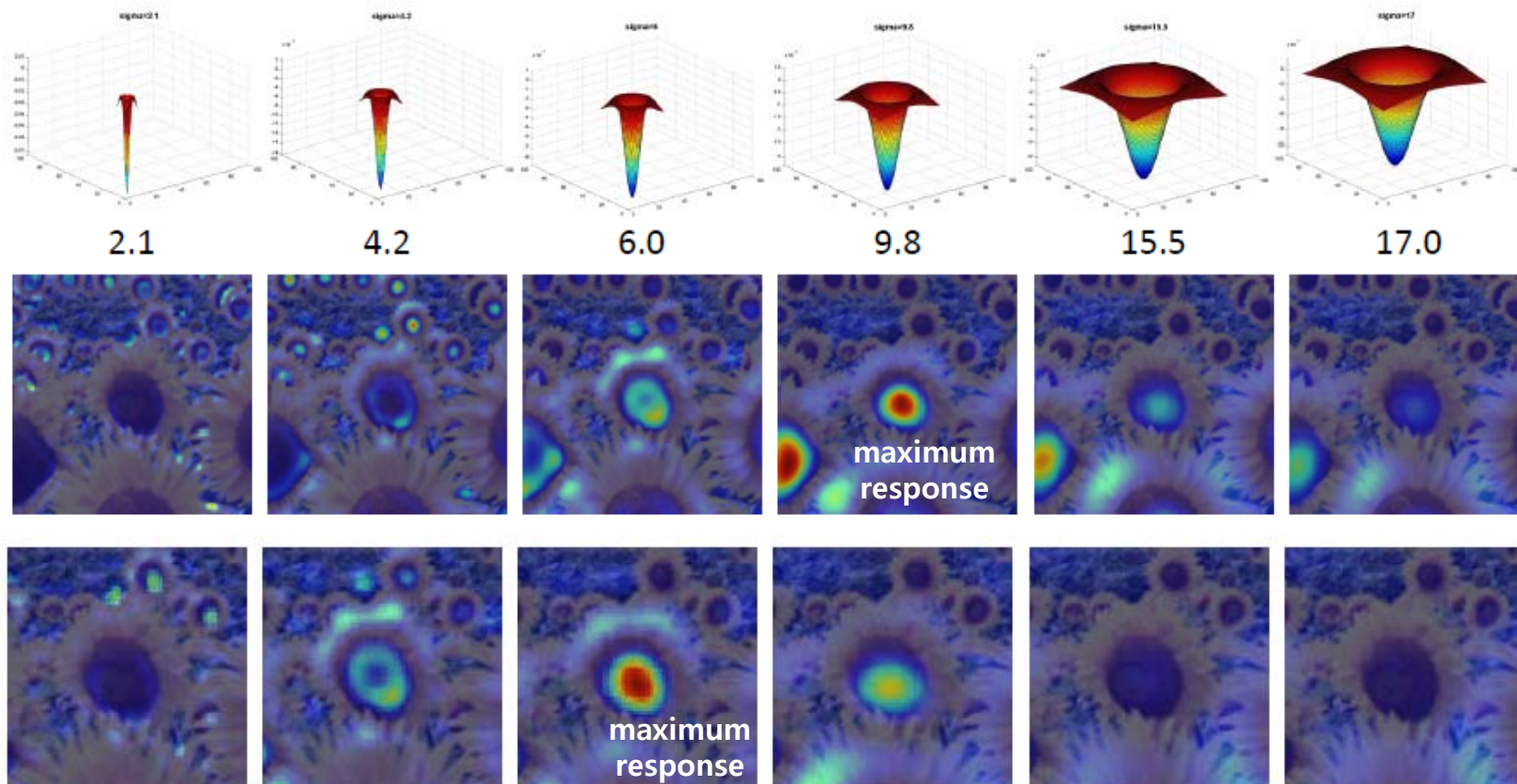


3/4 size

Full size

Blob detection

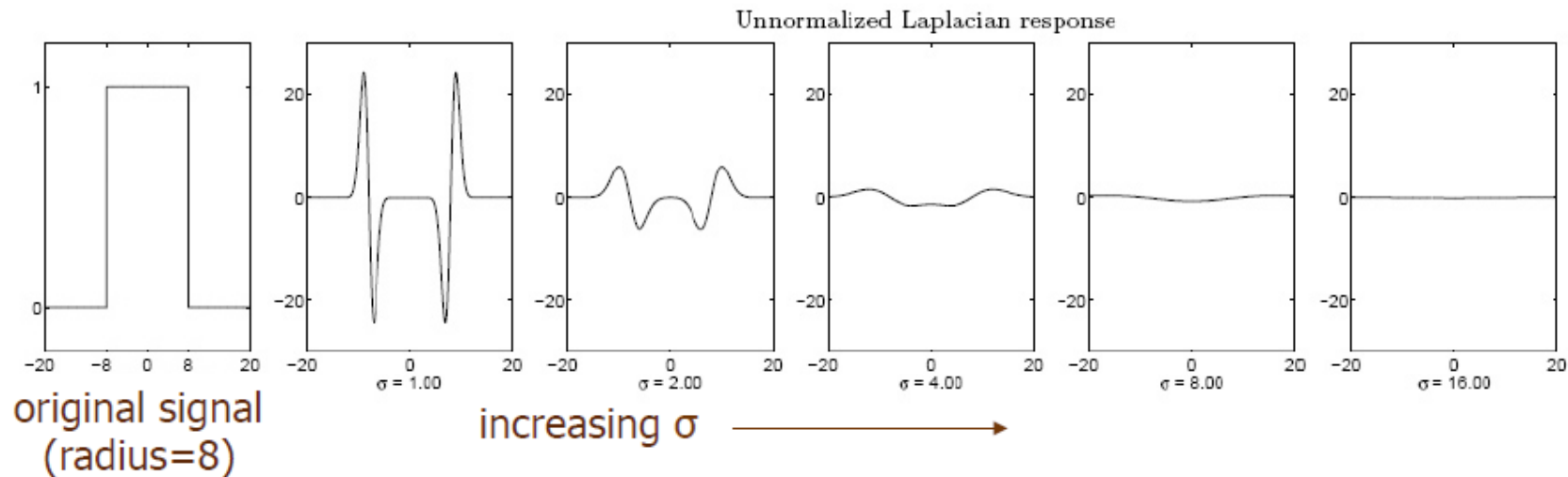
■ Example



Blob detection

■ Scale selection

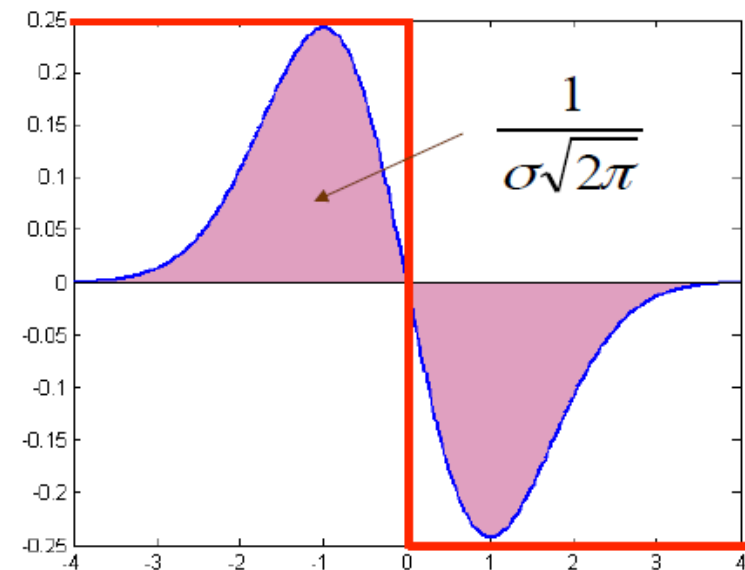
- We want to find the **characteristic scale** of the blob by convolving it with Laplacians at several scales and looking for the maximum response
- However, Laplacian response decays as scale increases



Blob detection

■ Scale normalization

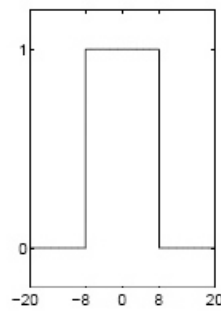
- The response of a derivative of Gaussian filter to a perfect step edge decreases as σ increases
- To keep response the same (scale-invariant), must multiply Gaussian derivative by σ
- Laplacian is the second Gaussian derivative, so it must be multiplied by σ^2



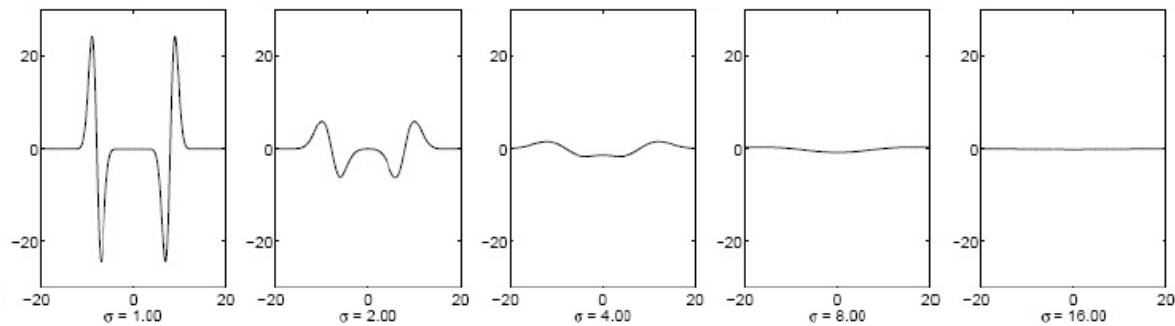
Blob detection

- Scale normalization

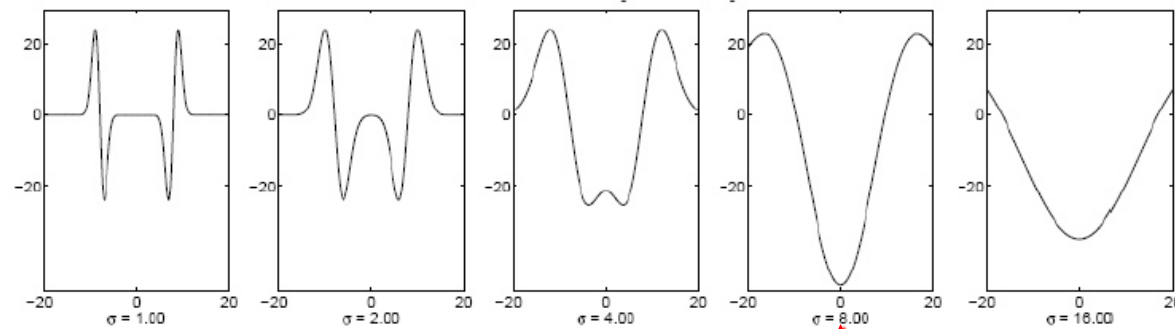
Original signal



Unnormalized Laplacian response



Scale-normalized Laplacian response



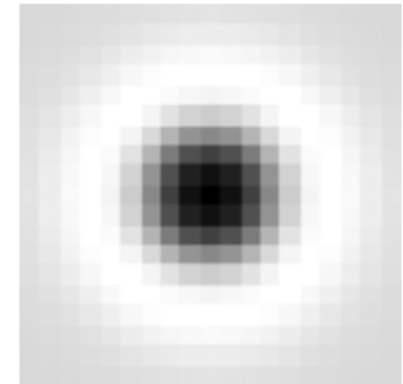
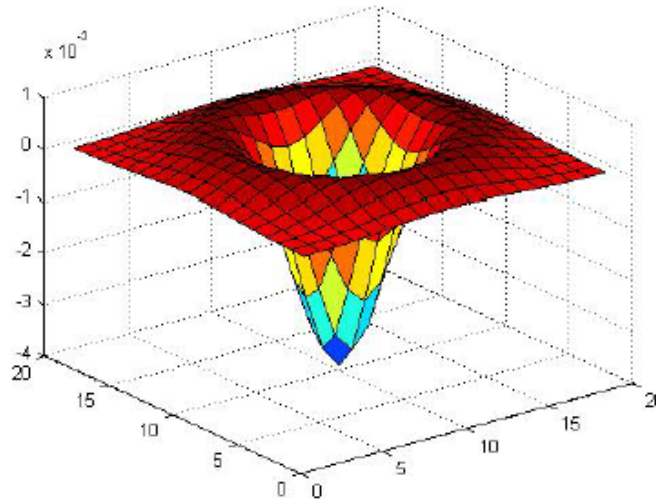
maximum

Blob detection

- Blob detection in 2D

Scale-normalized:

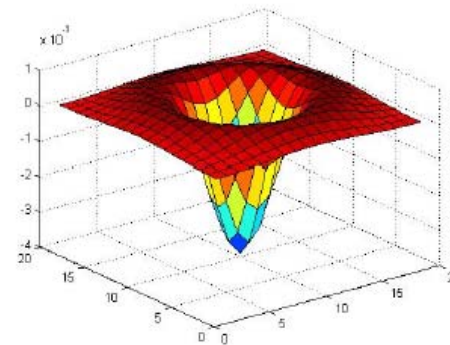
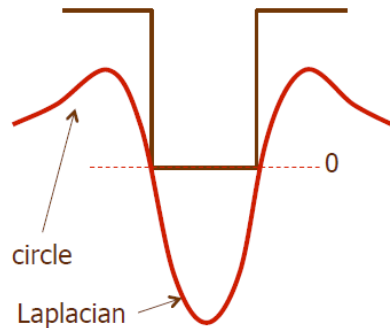
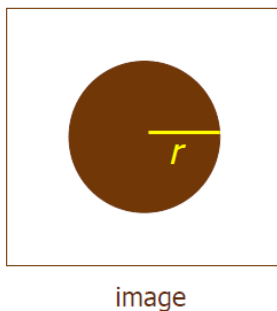
$$\nabla_{\text{norm}}^2 g = \sigma^2 \left(\frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2} \right)$$



Blob detection

■ Scale selection

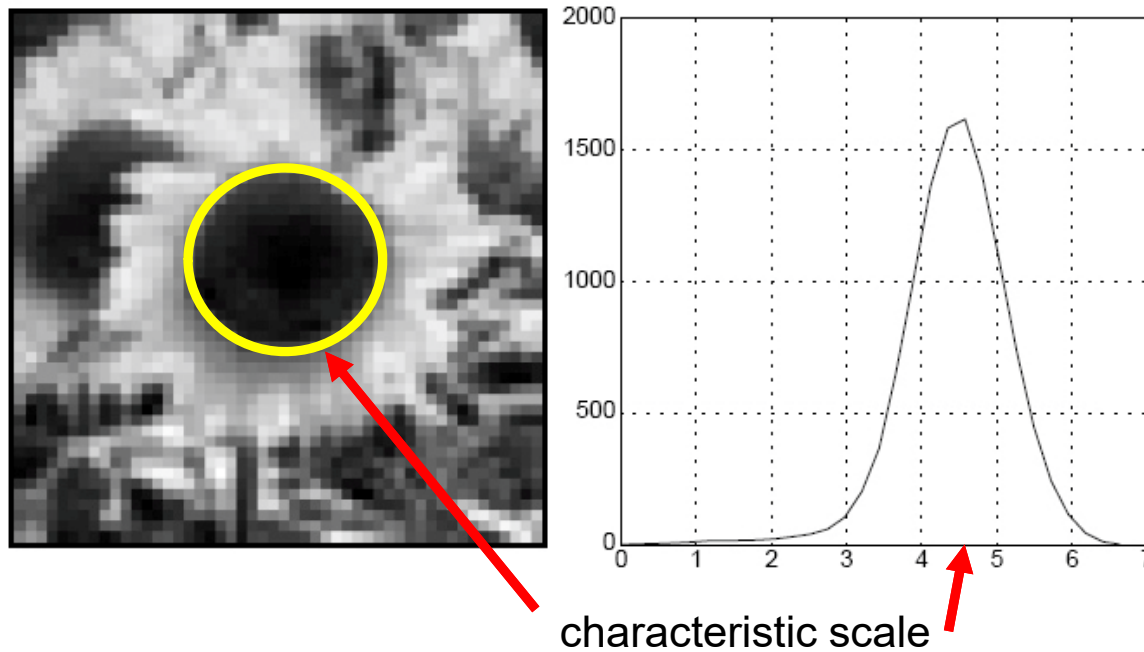
- At what scale does the Laplacian achieve a maximum response to a binary circle of radius r ?
- To get maximum response, the zeros of the Laplacian have to be aligned with the circle
- The Laplacian is given by (up to scale): $(x^2 + y^2 - 2\sigma^2) e^{-(x^2+y^2)/2\sigma^2}$
- Therefore, the maximum response occurs at $\sigma = r / \sqrt{2}$



Blob detection

- Characteristic scale

- We define the characteristic scale as the scale that produces peak of Laplacian response

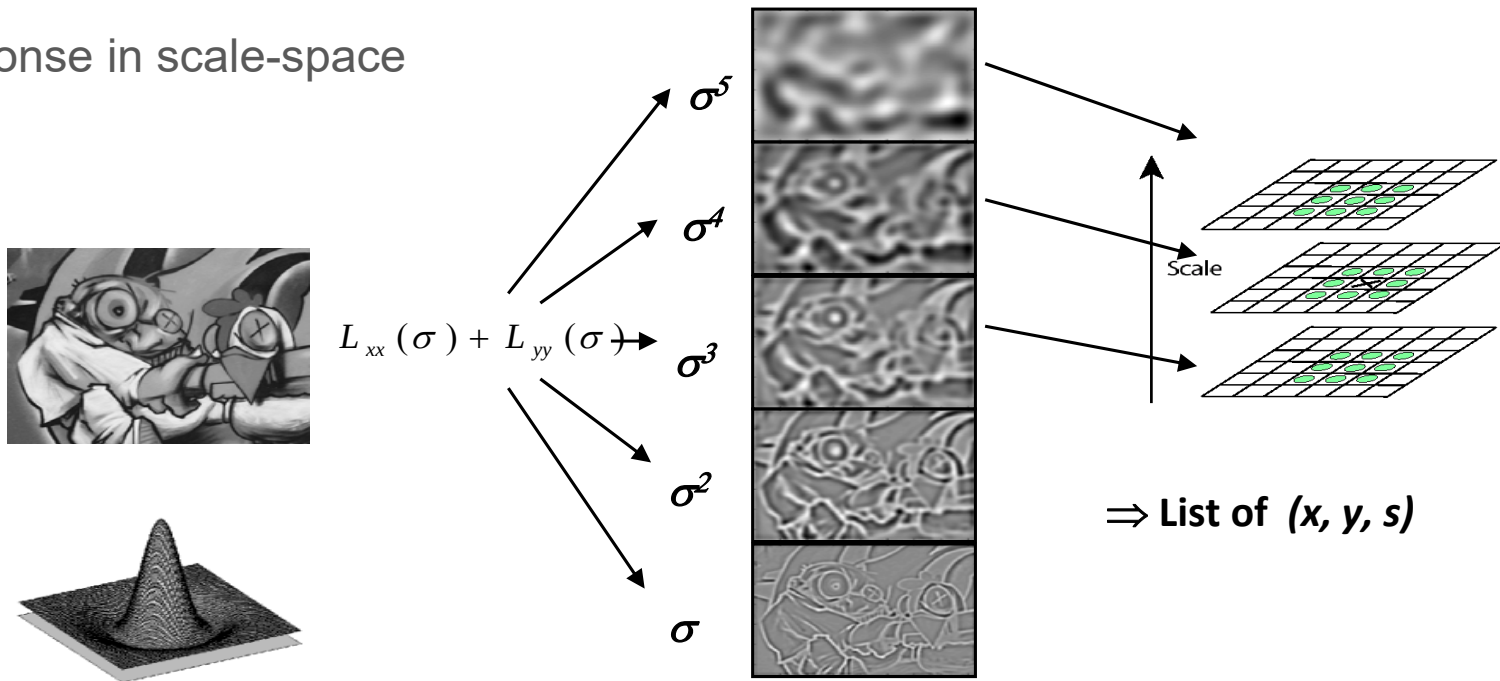


Blob detection

- Characteristic scale

1. Convolve image with scale-normalized Laplacian at several scales
2. Find maxima of squared Laplacian

response in scale-space



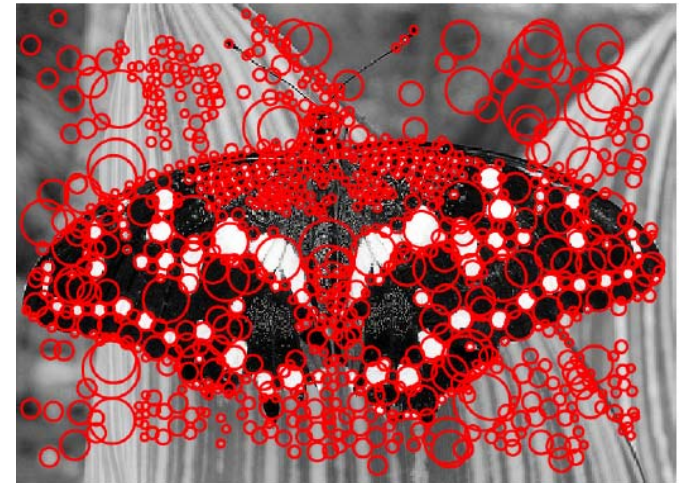
K. Grauman, B. Leibe

Blob detection

- Scale-space blob detector: example



$\text{Sigma} = 11.9912$



Blob detection

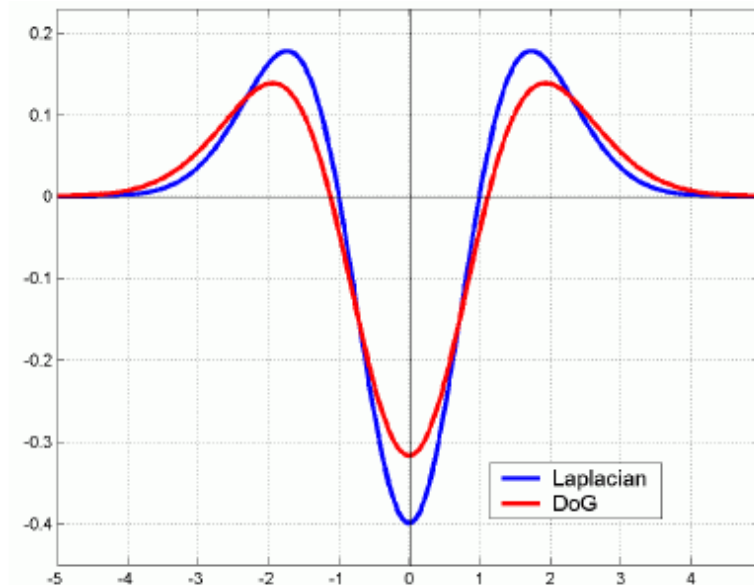
- Efficient implementation
 - Approximating the Laplacian with a difference of Gaussians:

$$L = \sigma^2 \left(G_{xx}(x, y, \sigma) + G_{yy}(x, y, \sigma) \right)$$

(Laplacian)

$$DoG = G(x, y, k\sigma) - G(x, y, \sigma)$$

(Difference of Gaussians)



Blob detection

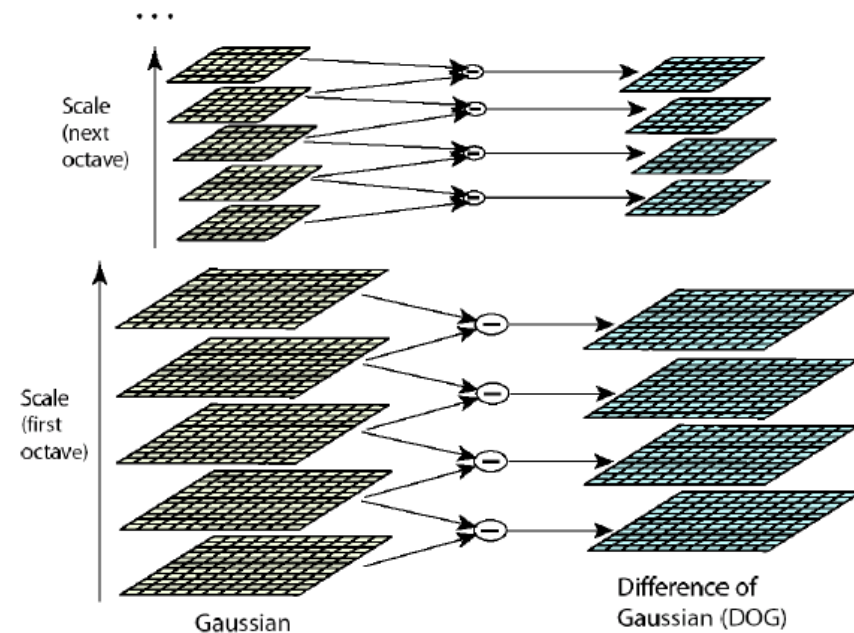
- Efficient implementation
 - Approximating the Laplacian with a difference of Gaussians:

$$L = \sigma^2 \left(G_{xx}(x, y, \sigma) + G_{yy}(x, y, \sigma) \right)$$

(Laplacian)

$$DoG = G(x, y, k\sigma) - G(x, y, \sigma)$$

(Difference of Gaussians)



Blob detection

- Invariance and covariance properties

- Covariant:

$$features(transform(image)) = transform(features(image))$$

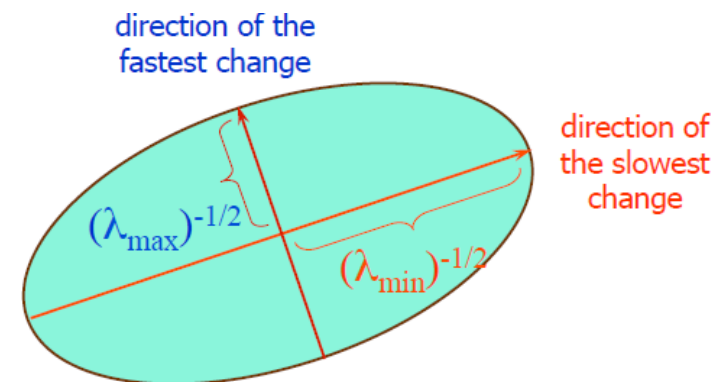
- Invariant:

$$features(transform(image)) = features(image)$$

- Blob location and scale is **covariant** w.r.t. rotation and scaling
- Laplacian (blob) response is **invariant** w.r.t. rotation and scaling

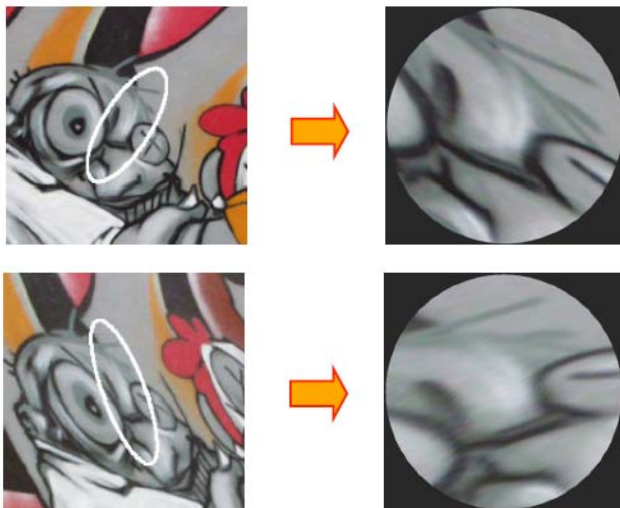
Blob detection

- Affine transformation approximates viewpoint changes for roughly planar objects and roughly orthographic cameras
- Consider the second moment matrix of the window containing the blob:
 - This ellipse visualizes the “characteristic shape” of the window



Blob detection

- From covariant detection to invariant description
 - Geometrically transformed versions of the same neighborhood will give rise to regions that are related by the same transformation
 - What to do if we want to compare the appearance of these image regions?
 - Normalization: transform these regions into same size circles



Problem: There is no unique transformation from an ellipse to a unit circle

Description is needed!!

Thanks!